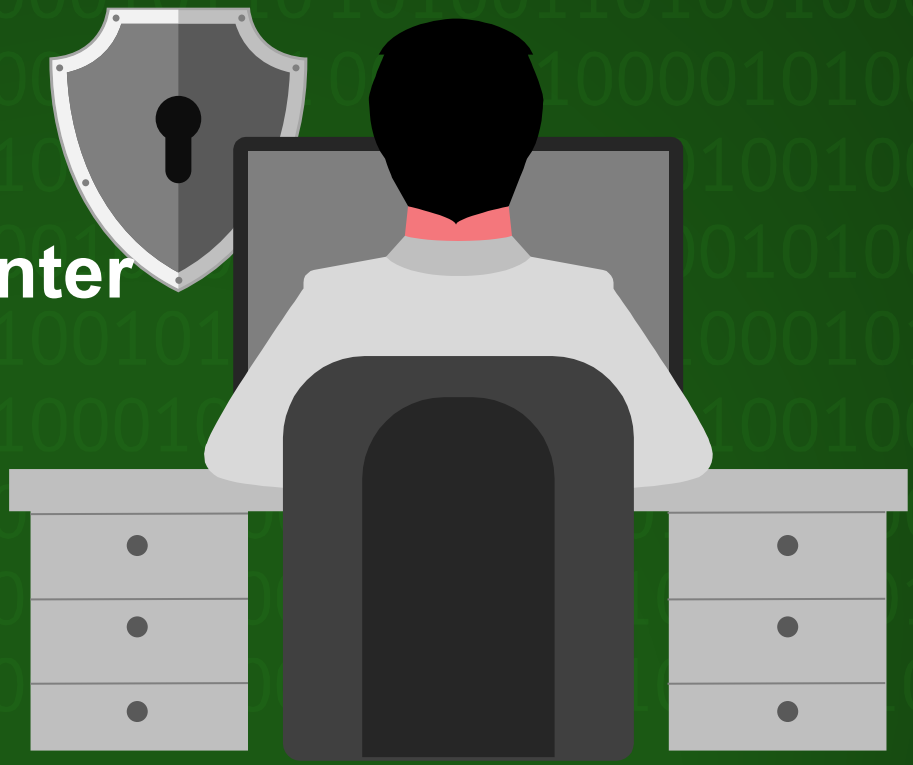
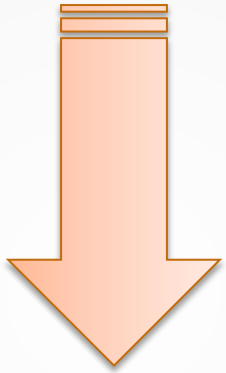
Computer Security: Principles and Practice

Database and Data Center Security



Chapter 5

The NIST (National Institute of Standards and Technology) Internal/Interagency Report NISTIR 7298 Defines the Term Computer Security as Follows:



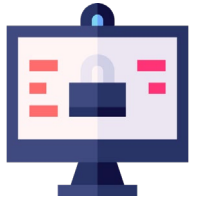
Measures and controls that ensure **Confidentiality**, **Integrity**, and **Availability** of information system **assets**.

These **assets** include Hardware, Software, Firmware, and information being processed, stored, and communicated.

WHAT IS THE WEAKEST POINT IN THE SYSTEM?

- Data is the Weakest part in any organization
- Encrypted communication
- Access to raw data in the HD
- Without the security, Databases will all collapse

Database Security



Complexity vs. Security – **Database Management Systems** (**DBMS**) are highly complex, but security techniques haven't evolved at the same pace.

SQL Vulnerabilities – **Structured Query Language (SQL)** is complex, and securing databases requires **deep knowledge of its weaknesses**.

Lack of Dedicated Security Staff – Most organizations do not have full-time **database security experts**.

Diverse IT Environments – Enterprises use a mix of **databases, operating systems, and platforms**, making security more difficult.

Cloud Dependency – More companies rely on **cloud databases**, introducing new security challenges.

Databases

- Structured collection of data stored for use by one or more applications
- Contains the **relationships** between **data items and groups of data items**
- Can sometimes contain **sensitive data** that needs to be **secured**
- **Query language**
  - Provides a uniform interface to the database **for users and applications**

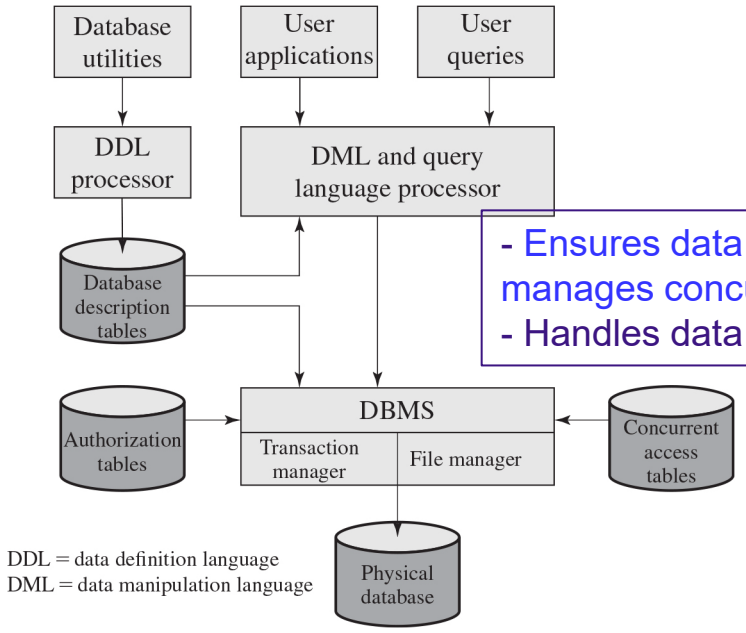
- **Database management system (DBMS)**
  - Group of programs for constructing and maintaining the database
  - Offers ad hoc (*one-time, spontaneous*) **query facilities** to multiple **users and applications**

Figure 5.1 DBMS Architecture

Users interact via applications, queries, and database utilities for data management.

Processes schema definitions and updates database description tables.

Handles user queries and application requests



- Ensures data consistency and manages concurrent transactions.
- Handles data storage and retrieval.

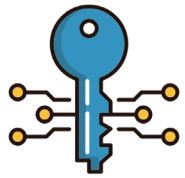
Control access permissions

Manage multi-user access

Stores actual data.

Relational Databases

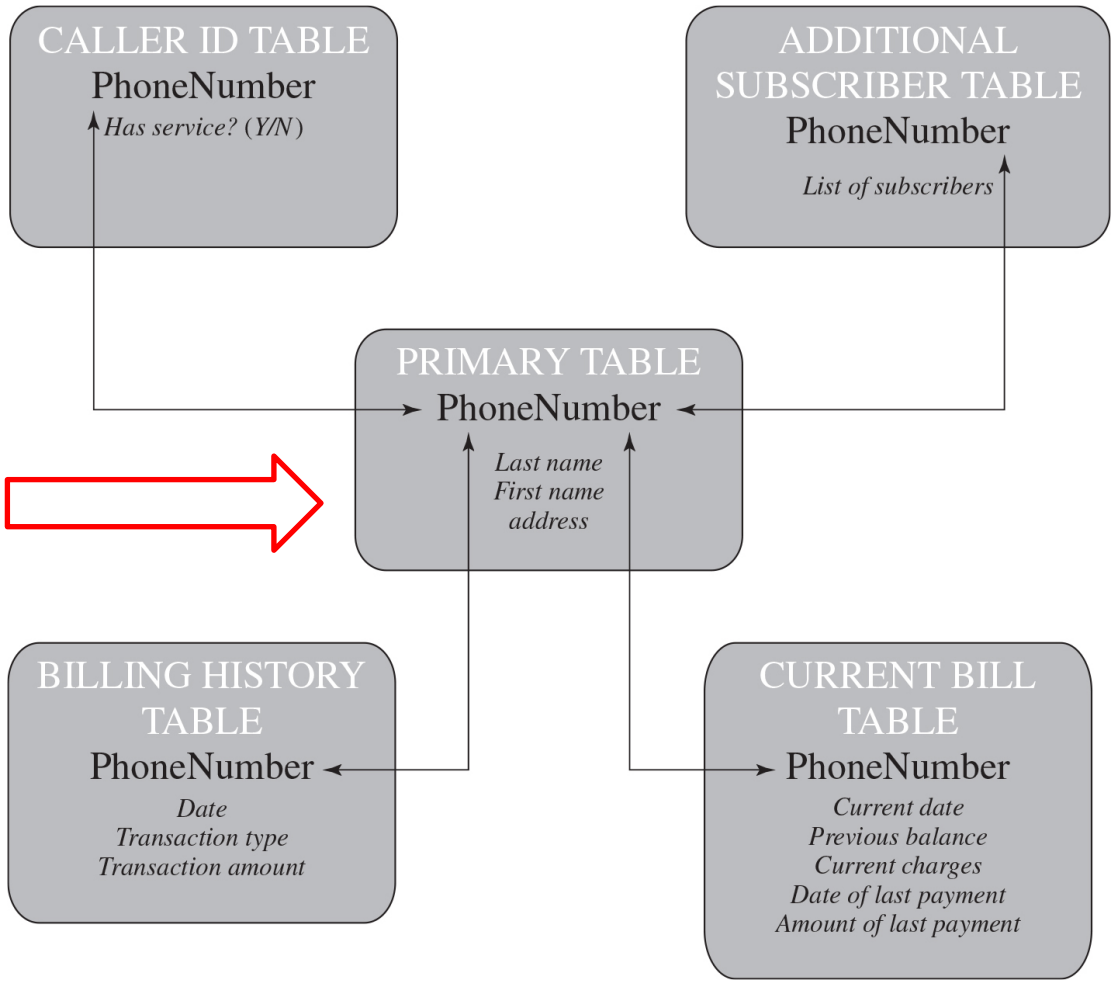
- Table of data consisting of rows and columns
  - Each column holds a particular type of data
  - Each row contains a specific value for each column
  - **Ideally has one column where all values are unique, forming an identifier/key for that row**



- Enables the creation of **multiple tables linked together** by a **unique identifier that is present in all tables**
- Use a **relational query language** to access the database
  - Allows the user to request data that fit a given set of criteria

Figure 5.2 Example Relational Database Model

What is the unique key?



RELATIONAL DATABASE ELEMENTS

- **View/virtual table**
  - Result of a query that returns ***selected rows and columns*** from one or more tables
  - ***Views are often used for security purposes***
- **Primary key**
  - Uniquely identifies a row
  - Consists of one or more column names
- **Foreign key**
  - Links one table to **attributes** in another
- ***Formal Name***
  - *Common Name/Also known as*
- **Relation**
  - Table/file
- **Tuple**
  - Row/record
- **Attribute**
  - Column/field

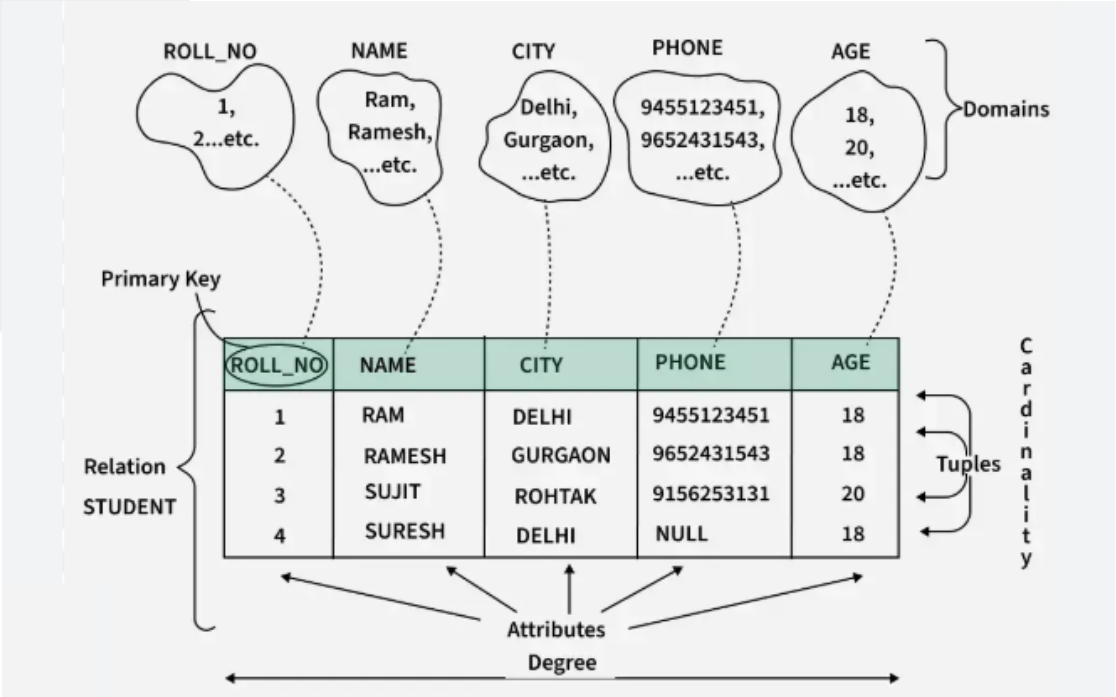
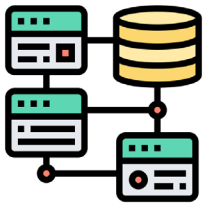
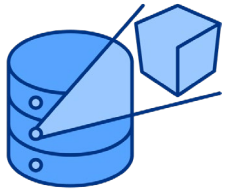


Figure 5.3 Abstract Model of a Relational Database





	Attributes									
	A_1	•	•	•	A_j	•	•	•	A_M
Records										
1	x_{11}	•	•	•	x_{1j}	•	•	•	x_{1M}
•	•				•				•
•	•				•				•
•	•				•				•
i	x_{i1}	•	•	•	x_{ij}	•	•	•	x_{iM}
•	•				•				•
•	•				•				•
•	•				•				•
N	x_{N1}	•	•	•	x_{Nj}	•	•	•	x_{NM}

Figure 5.4 Relational Database Example

- **Primary key**
  - Uniquely identifies a row
  - Consists of one or more column names
- **Foreign key**
  - Links one table to **attributes** in another
- **View/virtual table**
  - Result of a query that returns ***selected rows and columns*** from one or more tables
  - ***Views are often used for security purposes***
  - Transactions should be encrypted and authenticated

Employee Table

Ename	Did	Salarycode	Eid	Ephone
Robin	15	23	2345	6127092485
Neil	13	12	5088	6127092246
Jasmine	4	26	7712	6127099348
Cody	15	22	9664	6127093148
Holly	8	23	3054	6127092729
Robin	8	24	2976	6127091945
Smith	9	21	4490	6127099380

Foreign key — Primary key

Department Table

Did	Dname	Dacctno
4	human resources	528221
8	education	202035
9	accounts	709257
13	public relations	755827
15	services	223945

Primary key

Dname	Ename	Eid	Ephone
human resources	Jasmine	7712	6127099348
education	Holly	3054	6127092729
education	Robin	2976	6127091945
accounts	Smith	4490	6127099380
public relations	Neil	5088	6127092246
services	Robin	2345	6127092485
services	Cody	9664	6127093148

Structured Query Language (SQL)

- ***Standardized language to define schema, manipulate, and query data in a relational database***
- Several similar versions of ANSI/ISO standard
- All follow the same basic syntax and semantics

- **SQL statements can be used to:**
  - ***Create tables***
  - ***Insert and delete data in tables***
  - ***Create views***
  - ***Retrieve data with query statements***

SQL Injection Attacks (SQLi)

SQLi

SQL injection (SQLi) refers to an attack where a cybercriminal attempts to use an input field to write and run malicious SQL statements.

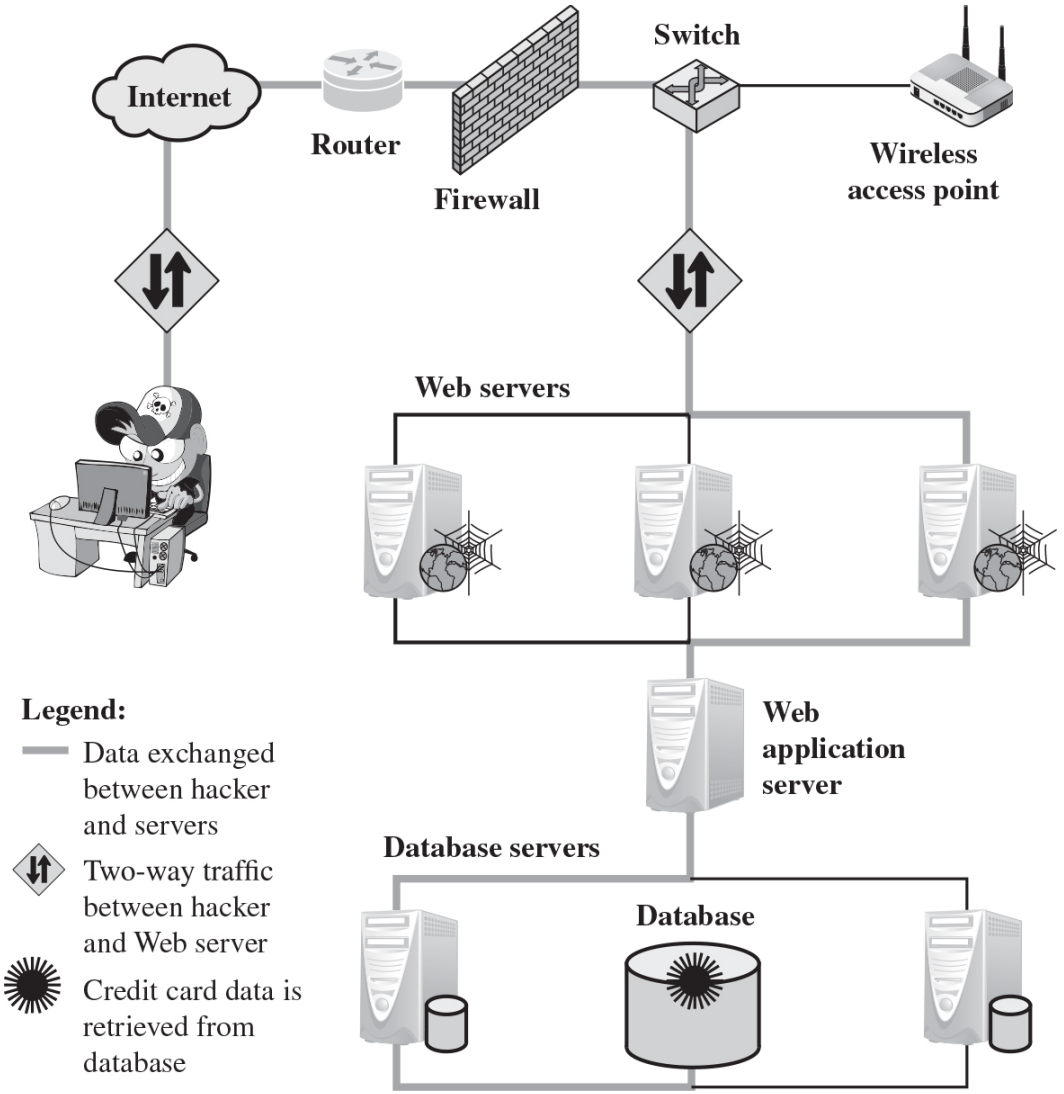
SQL Injection Attacks (SQLi)

- One of the most prevalent and dangerous network-based security threats
- Designed to exploit the nature of Web application pages
- **Sends malicious *SQL commands* to the database server**
- Most common attack goal ***is bulk extraction of data***
- Depending on the environment, SQL injection can also be exploited to:
  - ***Modify or delete data***
  - ***Execute arbitrary operating system commands***
  - ***Launch denial-of-service (DoS) attacks***

Figure 5.5 Typical SQL Injection Attack

2) The Web server forwards the malicious code to the Web application server.

1) A hacker exploits a vulnerability in a Web application to inject an SQL command, bypassing the firewall.



3) The Web application server sends the code to the database server.

4) The database executes the code, retrieving credit card data.

5) The Web application server generates a page displaying the stolen data.

6) The Web server delivers the credit card details to the hacker.

INJECTION TECHNIQUE

- ***The SQLi attack typically works by prematurely terminating a text string and appending a new command***
  - Because the inserted command may have additional strings appended to it before it is executed the attacker terminates the injected string with a comment mark "**- -**"
- ***Subsequent text is ignored at execution time***

Example of SQL Injection

Scenario

A web application has a login form where users enter their username and password. The backend SQL query might look like this:

sql Copy Edit

```sql
SELECT * FROM users WHERE username = 'user_input' AND password = 'password_input';
```

If an attacker inputs the following into the **username field**:

sql Copy Edit

```sql
admin' --
```

And leaves the password field empty, the SQL query becomes:

sql Copy Edit

```sql
SELECT * FROM users WHERE username = 'admin' -- ' AND password = '';
```

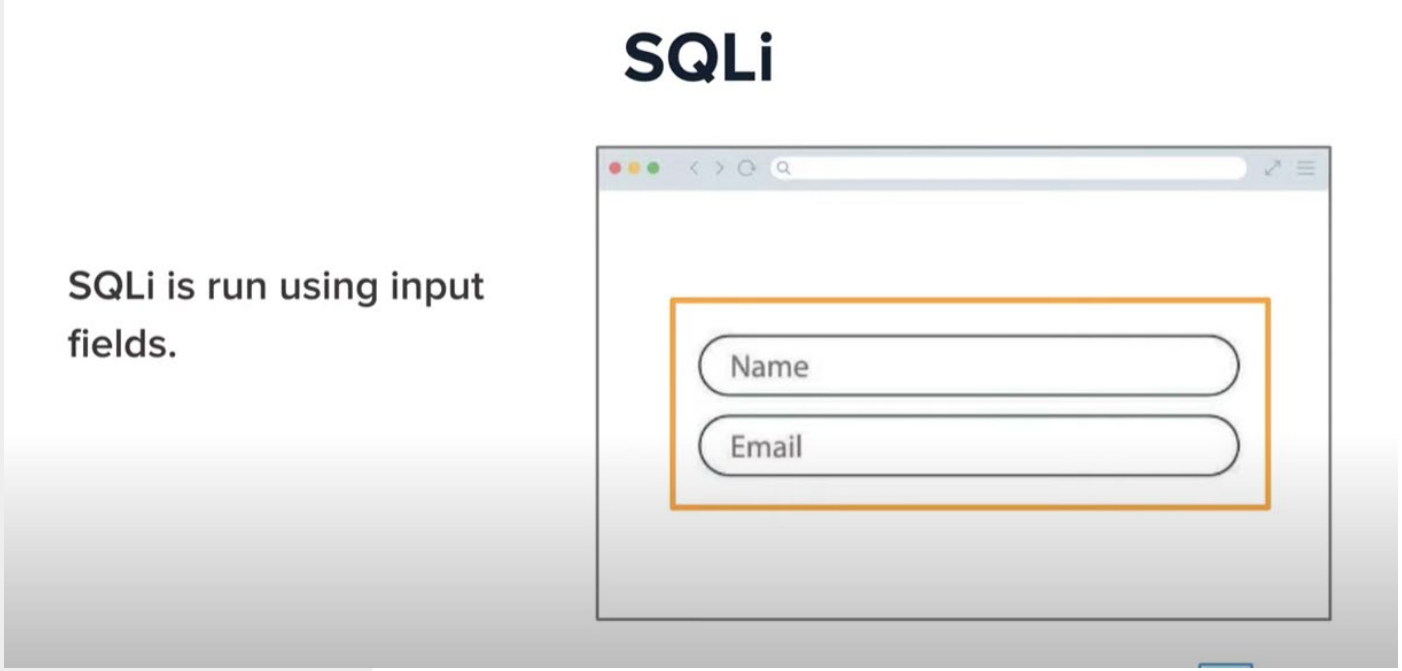
What Happens?

- The `--` comment marker ignores everything after it, including the password condition.
- The query now only checks for `username = 'admin'` and ignores password validation.
- If an account with the username `admin` exists, the attacker gains access **without needing a password.**

SQLI ATTACK AVENUES

1. User input

- Attackers inject SQL commands by providing suitable crafted user input



SQLI ATTACK AVENUES

2. Server variables

- Attackers can fake the values that are placed in HTTP and network headers and exploit this vulnerability by placing data directly into the headers

SQLI ATTACK AVENUES

3. Second-order injection

- **A malicious user** could rely on data already present in the system or database to trigger an SQL injection attack, so when the attack occurs, the input that modifies the query to cause an attack does not come from the user, but from within the system itself

SQLI ATTACK AVENUES

4. Cookies

- SQL Injection can be performed via cookies, an attacker could alter cookies such that when the application server builds an SQL query based on the cookie's content, the structure and function of the query is modified

How It Works:

- Attackers modify cookie values to inject malicious SQL.
- If the application constructs an SQL query using unsanitized cookie values, SQLi is possible.

Impact:

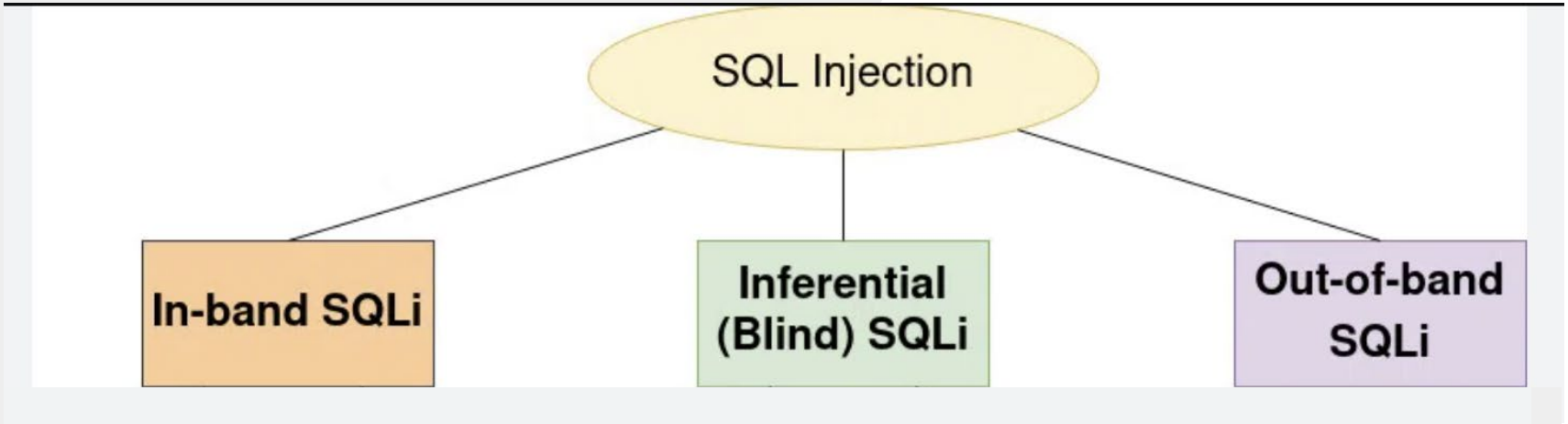
•Session hijacking (stealing active user sessions).
•Unauthorized access to other users' accounts.

SQLI ATTACK AVENUES

5. Physical user input

- Applying user input that constructs an attack outside the realm of web requests
- Example: exploiting barcode or QR code scanners.
  - If the scanner appends the input directly to an SQL query, an attacker can embed malicious SQL inside a custom barcode or QR code.

TYPES OF SQLI ATTACKS



Same Communication Channel

1. Tautology Attack
2. End-f-Line Comment
3. Piggybacked Queries

Different Communication Channels

INBAND ATTACKS – TAUTOLOGY ATTACK

- Uses the **same communication channel** for **injecting SQL** code and retrieving results
- The retrieved data are presented directly in application Web page
- Include:

1. Tautology

  - This form of attack injects code **in one or more conditional statements** so that they always evaluate to true
  - **Bypasses authentication** and logs the attacker in as the first user in the database (often an admin).

1. Tautology Attack

Definition: This attack injects code into a conditional statement so that it **always evaluates to TRUE**, allowing unauthorized access.

Example:

Consider a login query like:

```sql
SELECT * FROM users WHERE username = 'user_input' AND password = 'password_input';
```

Attacker Input:

Username:

```sql
' OR 1=1 --
```

Password: *(any value or left blank)*

Resulting Query:

```sql
SELECT * FROM users WHERE username = '' OR 1=1 -- ' AND password = '';
```

- `OR 1=1` **always evaluates to TRUE.**
- The `--` **comment marker ignores the rest of the query** (password check).
- The attacker **logs in without a password.**

INBAND ATTACKS - END-OF-LINE COMMENT

2. End-of-line comment

- After injecting code into a particular field, legitimate code that follows are **nullified** through usage of **end of line comments**
- This attack uses SQL comment symbols (-- or #) to ignore the rest of the query. It is often used to modify logic in authentication checks.

How It Works:

- Attackers input a value followed by a comment (--) to truncate the query.
- This removes security conditions (like password checks).

Why It Works?

- The -- ignores everything after it, including the password check.
- The database returns the admin account, allowing unauthorized access

Impact:

- Bypasses authentication.
- Allows access to restricted sections of the application.

2. End-of-Line Comment Attack

Definition: The attacker **nullifies** the rest of the SQL statement by injecting a comment (`--`), bypassing security checks.

Example:

A query checks user roles before allowing access:

```sql
SELECT * FROM users WHERE username = 'user_input' AND password = 'password_input' AND role
```

Attacker Input:

Username:

```sql
admin' --
```

Password: *(any value or left blank)*

Resulting Query:

```sql
SELECT * FROM users WHERE username = 'admin' -- ' AND password = '' AND role = 'admin';
```

- The `--` **removes the password and role check**, granting access as "admin."

INBAND ATTACKS - PIGGYBACKED QUERIES

3. Piggybacked queries

- The attacker adds **additional queries beyond the intended query**, piggy-backing the attack on top of a legitimate request.
- This attack injects multiple SQL statements into a single input field. The database executes all queries sequentially, allowing unauthorized operations.

How It Works:

- The attacker "piggybacks" an additional malicious SQL query after the legitimate query.
- This technique works only if multiple queries are allowed in a single execution.

Impact:

- Attackers can **delete tables, modify data, or escalate privileges**.
- Can **drop entire databases**, making recovery difficult.

3. Piggybacked Queries Attack

Definition: The attacker **adds an additional query** after the original one, executing malicious commands.

Example:

A normal query updates user details:

```sql
UPDATE users SET last_login = NOW() WHERE username = 'user_input';
```

Attacker Input:

Username:

```sql
'; DROP TABLE users; --
```

(No password needed)

Resulting Query:

```sql
UPDATE users SET last_login = NOW() WHERE username = ''; DROP TABLE users; -- ';
```

- The first statement **completes normally.**
- The `DROP TABLE users;` **deletes the entire users table.**
- The `--` **prevents errors by ignoring the rest.**

INFERENTIAL ATTACK

- *There is no actual transfer of data, but the attacker is able to reconstruct the information by sending particular requests and observing the resulting behavior of the Website/database server*
- Include:
  - Illegal/logically incorrect queries
    - This attack lets an attacker gather important information about the type and structure of the backend database of a Web application
    - The attack is considered a preliminary, information-gathering step for other attacks
  - Blind SQL injection
    - Allows attackers to infer the data present in a database system even when the system is sufficiently secure to not display any erroneous information back to the attacker

FIGURE 5.7 INDIRECT INFORMATION ACCESS VIA INFERENCE CHANNEL

Inference - is the process of performing authorized queries and deducing unauthorized information from the legitimate responses received.

The inference problem arises when the combination of a number of data items is more sensitive than the individual items, or when **a combination of data items can be used to infer data of a higher sensitivity.**

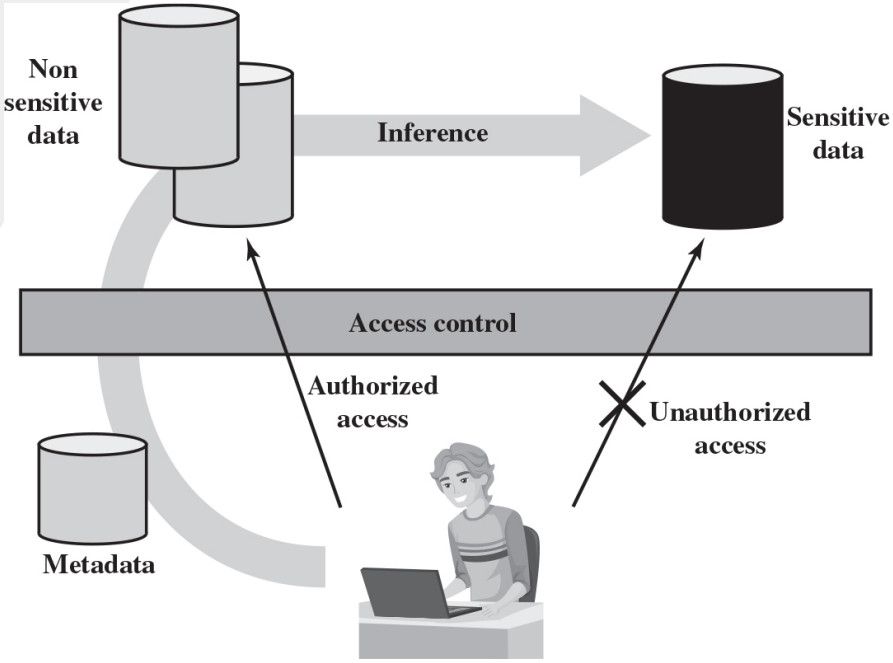


Figure 5.8 Inference Example (1 of 2)

(a) Inventory table

Item	Availability	Cost ($)	Department
Shelf support	in-store/online	7.99	hardware
Lid support	online only	5.49	hardware
Decorative chain	in-store/online	104.99	hardware
Cake pan	online only	12.99	housewares
Shower/tub cleaner	in-store/online	11.99	housewares
Rolling pin	in-store/online	10.99	housewares

(b) Two views

Availability	Cost ($)
in-store/online	7.99
online only	5.49
in-store/online	104.99

Item	Department
Shelf support	hardware
Lid support	hardware
Decorative chain	hardware

Figure 5.8 Inference Example (2 of 2)

(c) Table derived from combining query answers

Item	Availability	Cost ($)	Department
Shelf support	in-store/online	7.99	hardware
Lid support	online only	5.49	hardware
Decorative chain	in-store/online	104.99	hardware

OUT-OF-BAND ATTACK

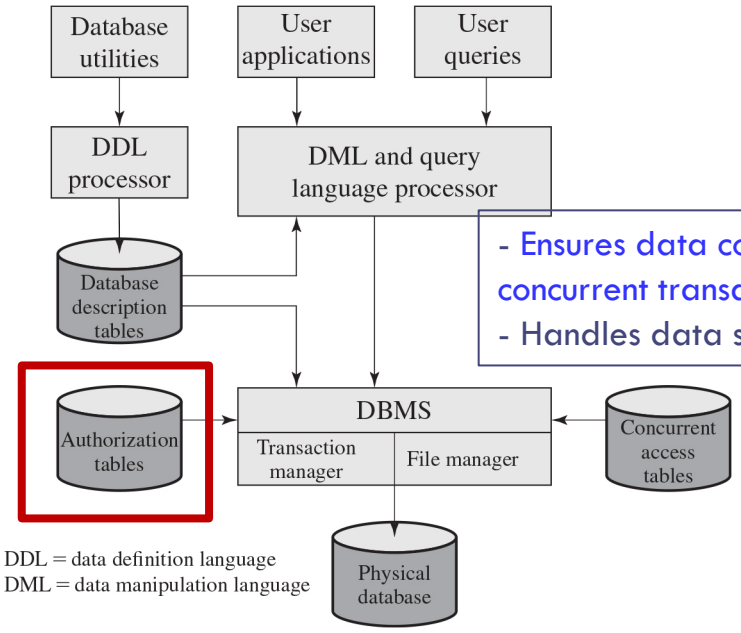
- **Data is retrieved using a different channel**
- This can be used when there are limitations on information retrieval, but outbound connectivity from the database server is lax

Figure 5.1 DBMS Architecture

Users interact via applications, queries, and database utilities for data management.

Processes schema definitions and updates database description tables.

Handles user queries and application requests



- Ensures data consistency and manages concurrent transactions.
- Handles data storage and retrieval.

Control access permissions

Manage multi-user access

Stores actual data.

DATABASE ACCESS CONTROL

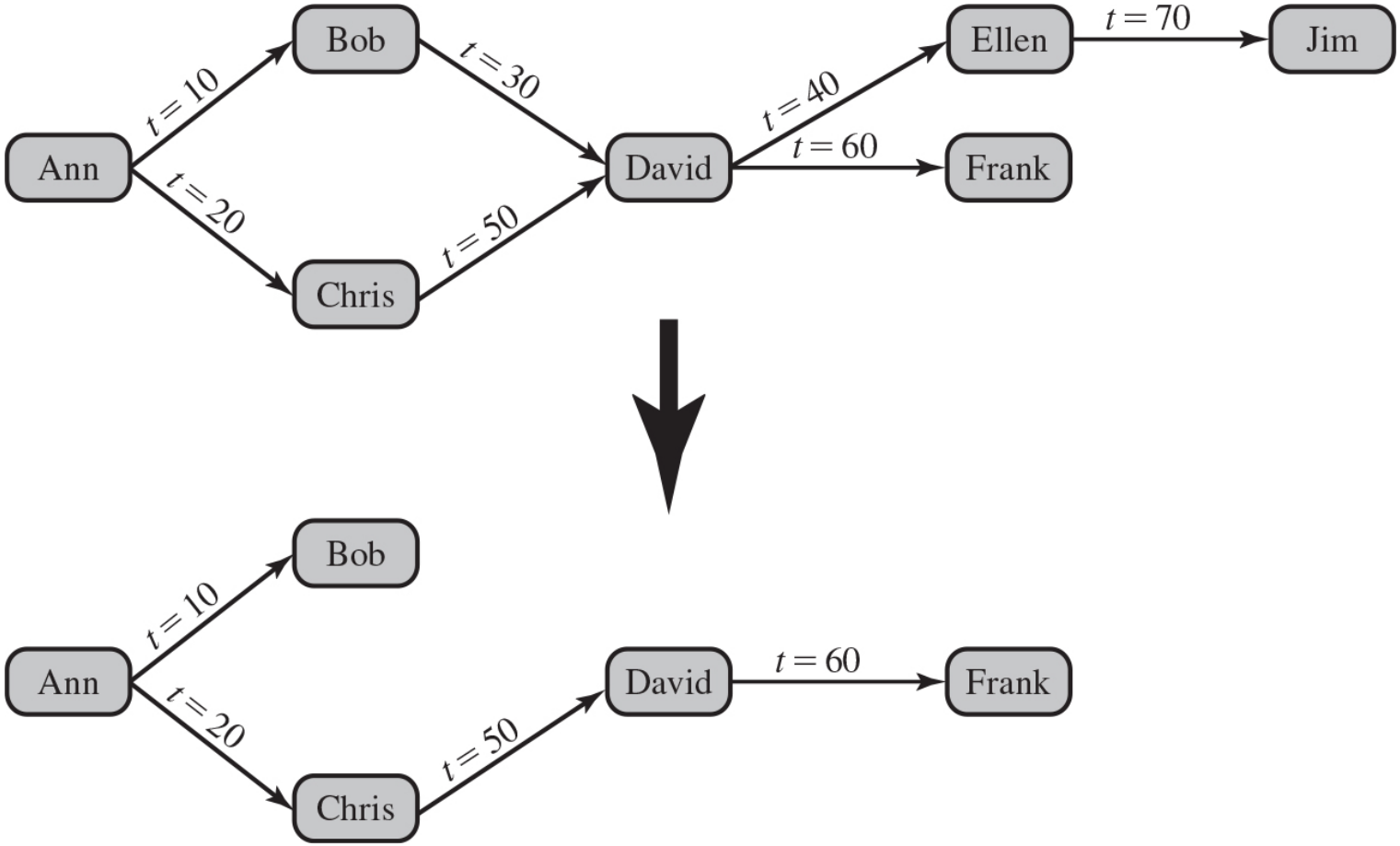
- Database access control system determines:
  - If the user has access to the entire database or just portions of it
  - What access rights the user has (create, insert, delete, update, read, write)

- Can support a range of administrative policies
  - **Centralized administration**
    - Small number of privileged users may grant and revoke access rights
  - **Ownership-based administration**
    - The creator of a table may grant and revoke access rights to the table
  - **Decentralized administration**
    - The owner of the table may grant and revoke authorization rights to other users, allowing them to grant and revoke access rights to the table

SQL ACCESS CONTROLS

- Two commands for managing access rights:
  - **Grant**
    - Used to grant one or more access rights or can be used to assign a user to a role
  - **Revoke**
    - Revokes the access rights
- **Typical access rights are:**
  - Select
  - Insert
  - Update
  - Delete
  - References

Figure 5.6 Bob Revokes Privilege From David



ROLE-BASED ACCESS CONTROL (RBAC)

- Role-based access control eases administrative burden and improves security
- A database **RBAC** needs to provide the following capabilities:
  - ***Create and delete roles***
  - ***Define permissions for a role***
  - ***Assign and cancel assignment of users to roles***

ROLE-BASED ACCESS CONTROL (RBAC)

- Categories of database users:
  - **Application owner**
    - An end user who **owns database objects** as part of an application
  - **End user**
    - An end user who **operates on database objects** via a particular application **but does not own any of the database objects**
  - **Administrator**
    - User who has **administrative responsibility** for part or all of the database

DATABASE ENCRYPTION

- The database is typically the most valuable information resource for any organization
  - **Protected by multiple layers of security**
    - Firewalls, authentication, general access control systems, DB access control systems, database encryption
    - **Encryption becomes the last line of defense in database security**
  - Can be applied to the entire database, at the record level, the attribute level, or level of the individual field

DATABASE ENCRYPTION

- **Disadvantages to encryption:**
  - **Key management**
    - Authorized users must have access to the decryption key for the data for which they have access
  - **Inflexibility**
    - When part or all the database is encrypted, it becomes more difficult to perform record searching
    - Computational resources

Selective Encryption – Sensitive information only

Figure 5.9 A Database Encryption Scheme

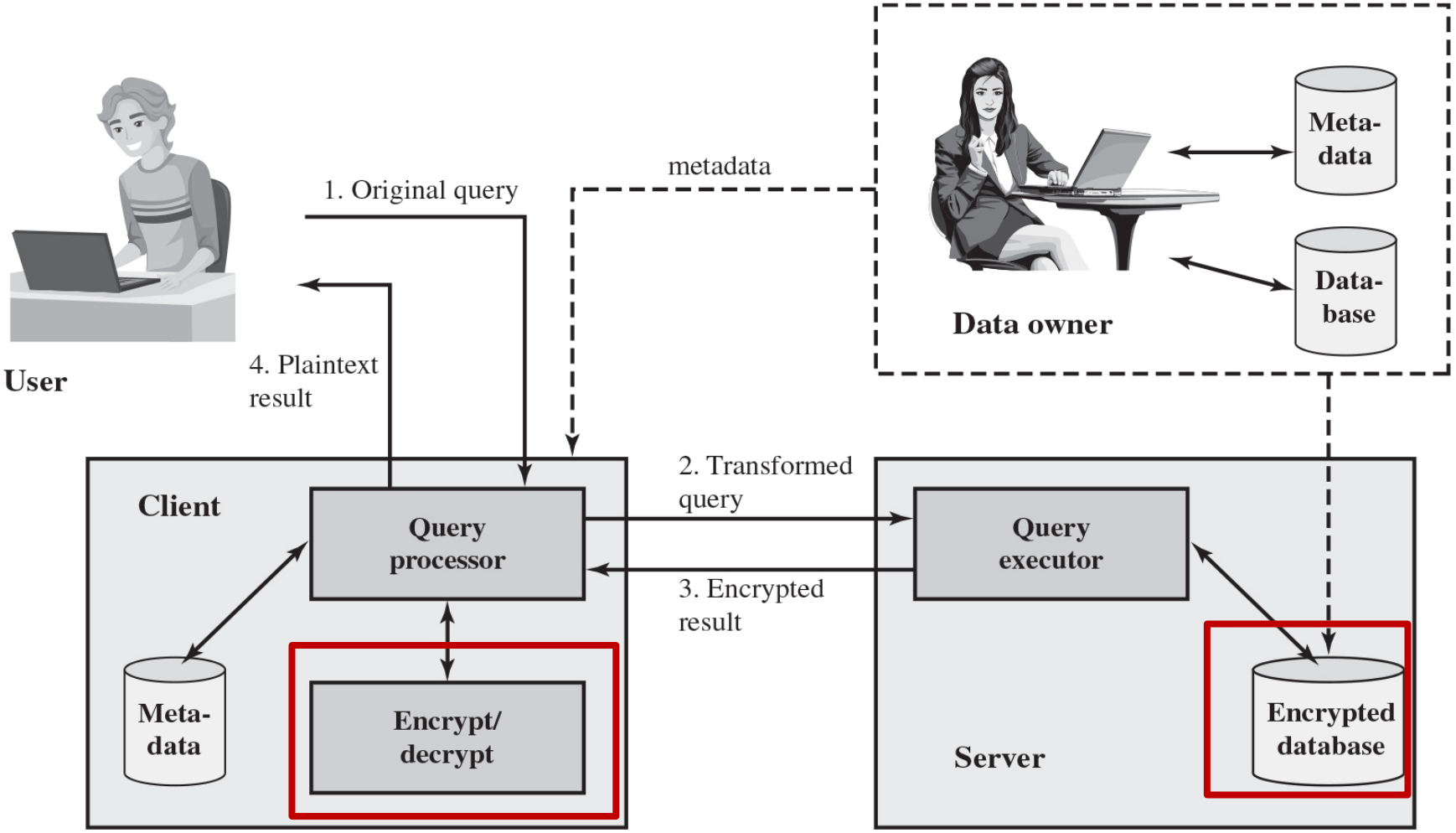
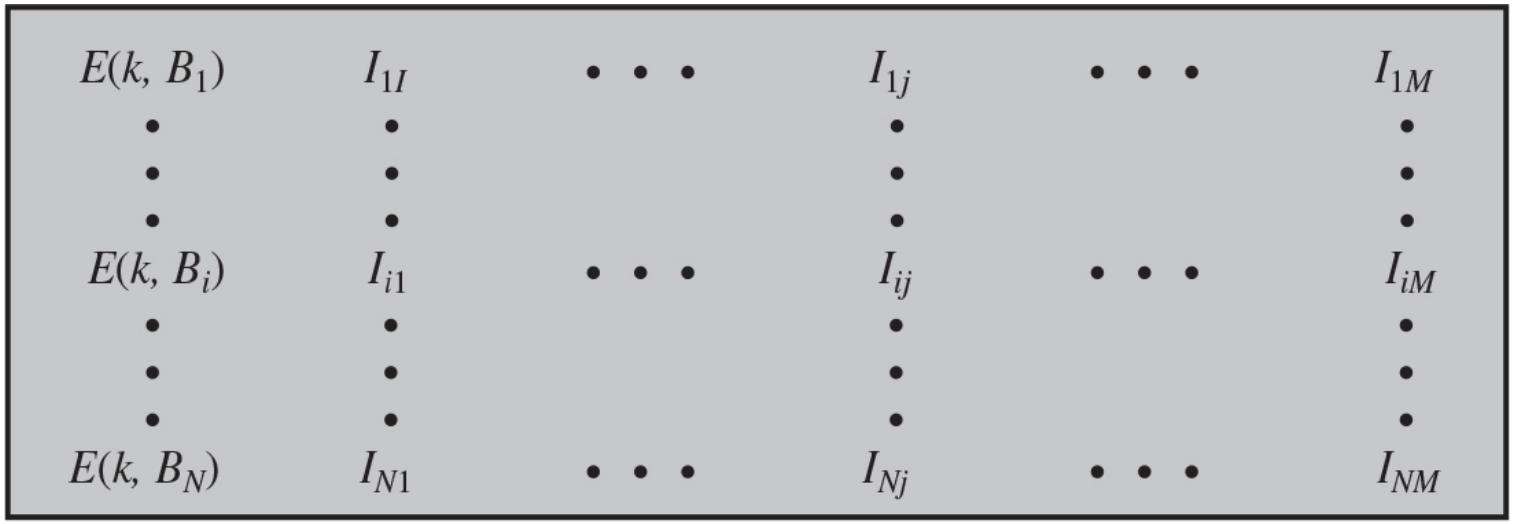


Figure 5.10 Encryption Scheme for Database of Figure 5.3



$B_i = (x_{i1} \| x_{i2} \| \ldots \| x_{iM})$

Table 5.3 Encrypted Database Example

(a) Employee Table

eid	ename	salary	addr	did
23	Tom	70K	Maple	45
860	Mary	60K	Main	83
320	John	50K	River	50
875	Jerry	55K	Hopewell	92

(b) Encrypted Employee Table with Indexes

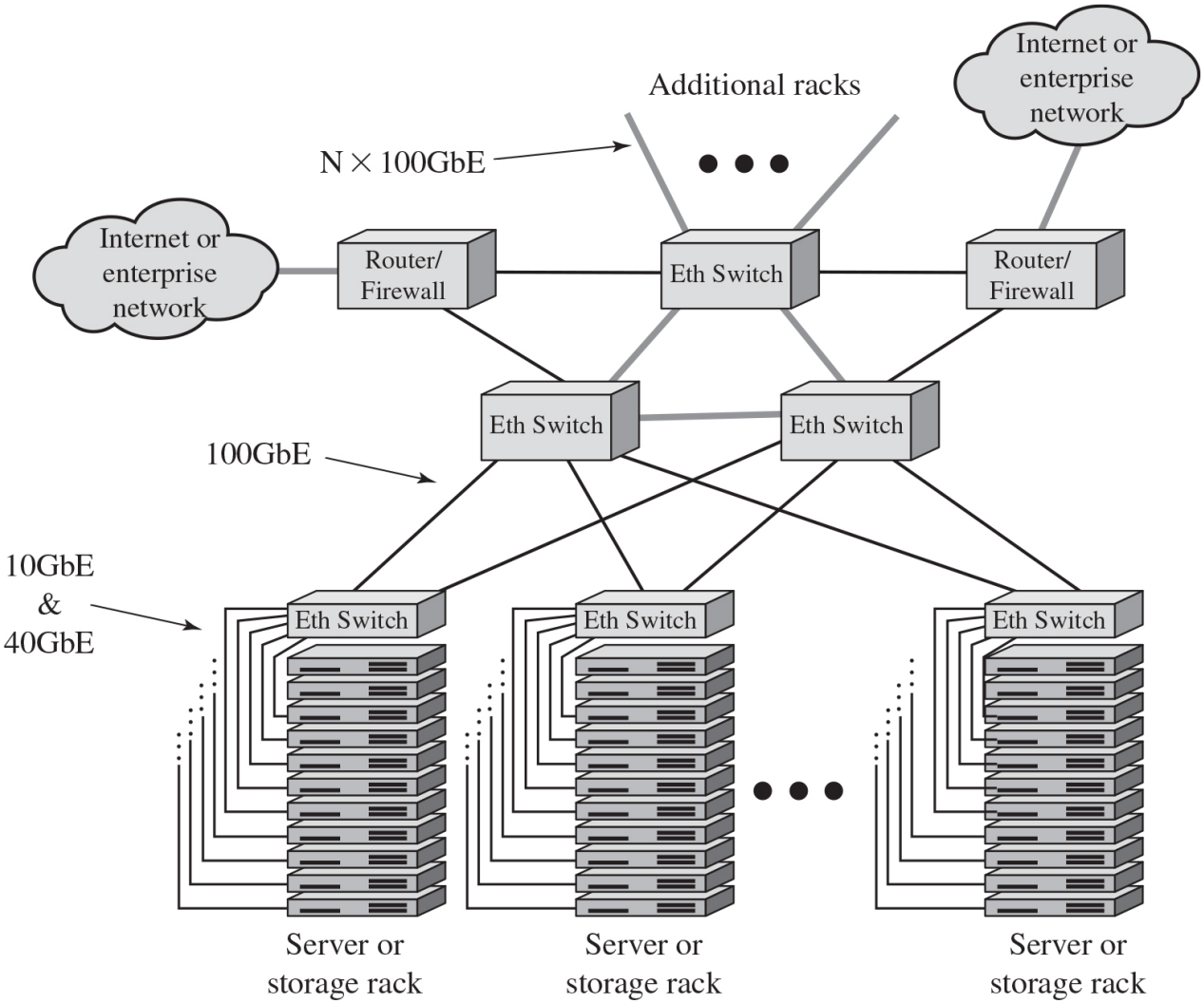
E(k, B)	I(eid)	I(ename)	I(salary)	I(addr)	I(did)
1100110011001011 . . .	1	10	3	7	4
0111000111001010 . . .	5	7	2	7	8
1100010010001101 . . .	2	5	1	9	5
0011010011111101 . . .	5	5	2	4	9

* addr: address, did: Department ID

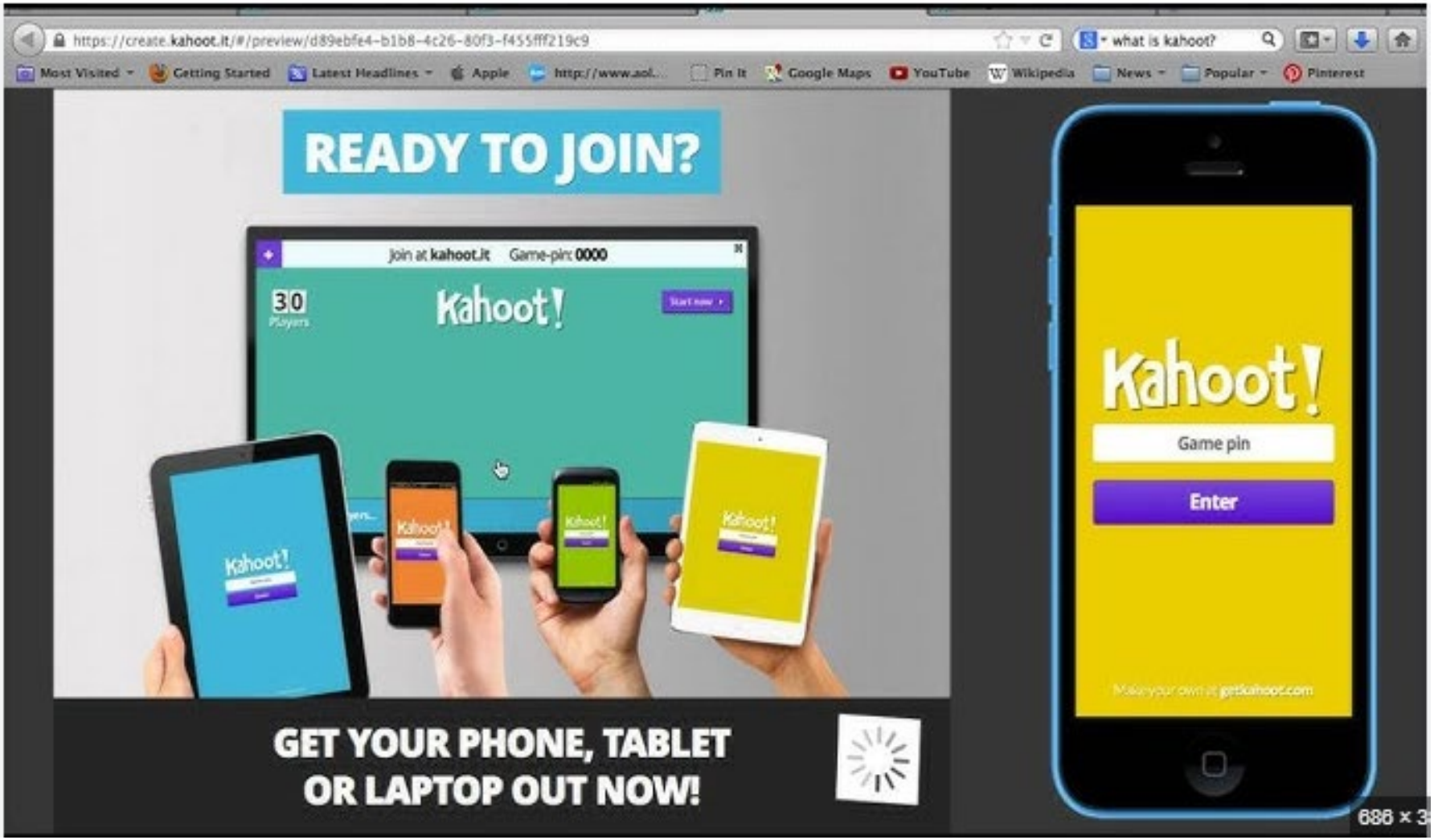
Data Center Security

- Data center:
  - An enterprise facility that houses a large number of servers, storage devices, and network switches and equipment
  - The number of servers and storage devices can run into the tens of thousands in one facility
  - **Generally, includes redundant or backup power supplies, redundant network connections, environmental controls, and various security devices**
  - Can occupy one room of a building, one or more floors, or an entire building
- Examples of uses include:
  - Cloud service providers
  - Search engines
  - Large scientific research facilities
  - IT facilities for large enterprises

Figure 5.11 Key Data Center Elements



Kahoot!


READY TO JOIN?
Join at kahoot.it Game-pin: 0000
30
Players
Kahoot!
Start now
Kahoot!
Game pin
Enter
GET YOUR PHONE, TABLET
OR LAPTOP OUT NOW!

1. What is the primary function of a Database Management System (DBMS)?
a) Hardware maintenance
b) Constructing and maintaining the database
c) Network security
d) Operating system management

2. In a relational database, a Primary Key is used to:
a) Encrypt data
b) Uniquely identify a row
c) Link multiple databases
d) Restrict data modification

3. Which query language is commonly used to interact with relational databases?
a) HTML
b) JavaScript
c) SQL
d) Python

4. What is a common goal of an SQL Injection (SQLi) attack?
a) Improve database efficiency
b) Extract sensitive data
c) Optimize queries
d) Perform load balancing

5. Which SQLi attack method uses conditional statements that always evaluate to true?
a) Piggybacked queries
b) Tautology
c) Blind SQL Injection
d) End-of-line comment

6. Which SQL statement is used to revoke access rights from a user?
a) DELETE
b) GRANT
c) REVOKE
d) REMOVE

7. What is the main purpose of Role-Based Access Control (RBAC) in databases?
a) To encrypt data
b) To ensure authorized users have proper access levels
c) To increase network speed
d) To improve data redundancy

8. What is a Foreign Key in a relational database?
a) A key used to encrypt data
b) A key that links one table to another
c) A key used to delete records
d) A backup key for authentication

9. Which of the following is NOT an avenue for SQL Injection attacks?
a) User input
b) Server variables
c) Encrypted files
d) Cookies

10. What is the last line of defense in database security?
a) Firewalls
b) Encryption
c) Password authentication
d) Access control

11. In SQL, which statement is used to create a new table?
a) CREATE TABLE
b) ADD TABLE
c) INSERT TABLE
d) BUILD TABLE

12. A View in a database is:
a) A temporary table used for security purposes
b) A new table with encrypted data
c) An external link to another database
d) A hardware component used in storage

13. Which type of attack does NOT transfer data but deduces information based on responses?
a) Out-of-band SQLi
b) Inferential SQLi
c) Inband SQLi
d) Tautology SQLi

14. What is the main disadvantage of database encryption?
a) It slows down the network
b) It makes searching data more difficult
c) It eliminates database access control
d) It requires additional firewalls

15. Which access control mechanism allows a table owner to grant access rights?
a) Centralized administration
b) Ownership-based administration
c) Decentralized administration
d) Encrypted access control

16. Which SQL command is used to modify existing records in a table?
a) CHANGE
b) UPDATE
c) EDIT
d) ALTER

17. What type of SQL Injection attack modifies the structure and function of queries using cookies?
a) User input SQLi
b) Second-order SQLi
c) Cookie-based SQLi
d) Server-variable SQLi

18. Which of the following is a key feature of Role-Based Access Control (RBAC)?
a) Granting access based on user identity alone
b) Assigning permissions to roles instead of individuals
c) Allowing all users full access to all database objects
d) Eliminating the need for authentication

19. In a database, which command is used to remove a table permanently?
a) REMOVE
b) DELETE
c) DROP TABLE
d) CLEAR

20. A data center primarily consists of:
a) Personal computers for small businesses
b) Large-scale servers, storage devices, and networking equipment
c) Single-user workstations
d) Physical backup tapes only

Summary

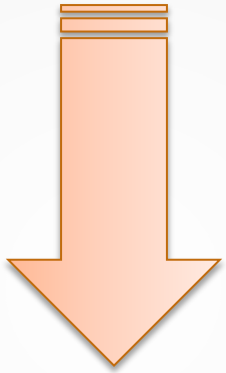
- The need for database security
- Database management systems
- Relational databases
  - Elements of a relational database system
  - Structured Query Language
- SQL injection attacks
  - A typical SQLi attack
  - The injection technique
  - SQLi attack avenues and types
  - SQLi countermeasures
- Database access control
  - SQL-based access definition
  - Cascading authorizations
  - Role-based access control
- Inference
- Database encryption

Computer Security: Principles and Practice

Internet Security Protocols and Standards

Chapter 22

The NIST (National Institute of Standards and Technology) Internal/Interagency Report NISTIR 7298 Defines the Term Computer Security as Follows:



Measures and controls that ensure **Confidentiality**, **Integrity**, and **Availability** of information system **assets**.

These **assets** include Hardware, Software, Firmware, and information being processed, stored, and communicated.

MIME and S/MIME

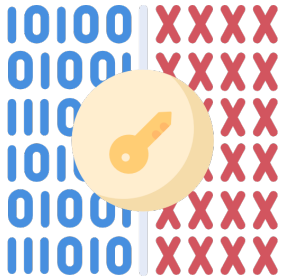
Protocols used to ensure secure and authentic emails

MIME

- **MIME (Multipurpose Internet Mail Extension)** is an **email extension protocol** that allows emails to **send multimedia content** such as images, audio, video, and attachments. It extends the capabilities of **RFC 822**, which originally only supported text messages.
- Extension to the old RFC 822 specification of an Internet mail format
  - RFC 822 defines a simple heading with To, From, Subject
  - Assumes ASCII text format
- Provides a number of new header fields that define information about the body of the message

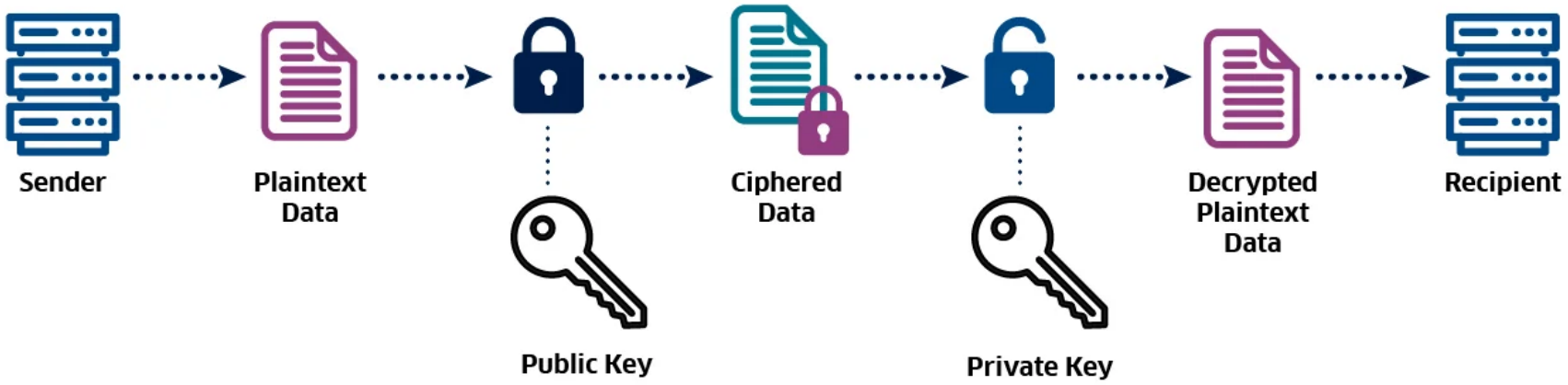
S/MIME

- **S/MIME (Secure Multipurpose Internet Mail Extension)** is an **enhanced version of MIME** that **adds encryption and digital signatures** to emails, ensuring security.



- Secure/**Multipurpose Internet Mail Extension**
- Security enhancement to the MIME Internet e-mail format
  - *Based on technology from RSA Data Security*
- Provides the ability to **sign** and/or **encrypt** e-mail messages

MIME and S/MIME



S/MIME Content Types

- **PKCS#7 (Public Key Cryptography Standards #7)**
  - PKCS#7 is a standard developed by RSA Security for cryptographic message syntax. **It defines how to package encrypted and signed data using public-key cryptography**.
- **PKCS7-MIME Formats**
  1) **SignedData** – Ensures email integrity and authentication.
  2) **EnvelopedData** – Provides encryption for secure email transmission.
  3) **CompressedData** – Compresses data before encryption/signing.

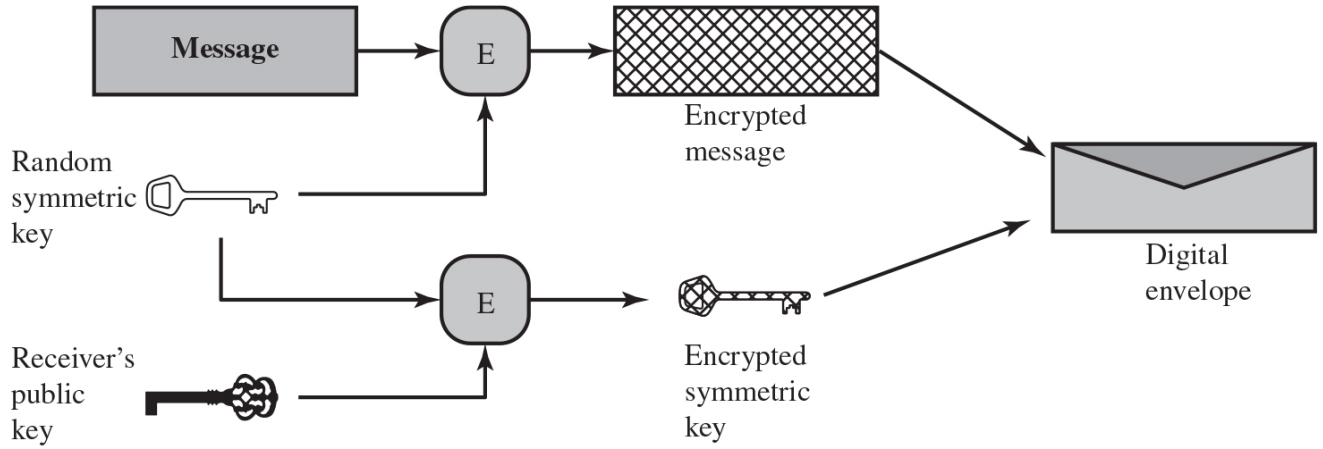
S/MIME Functions

Enveloped data	• Encrypted content and associated keys
Signed data	• Encoded message + signed digest
Clear-signed data	• Cleartext message + encoded signed digest
Signed and enveloped data	• Nesting of signed and encrypted entities

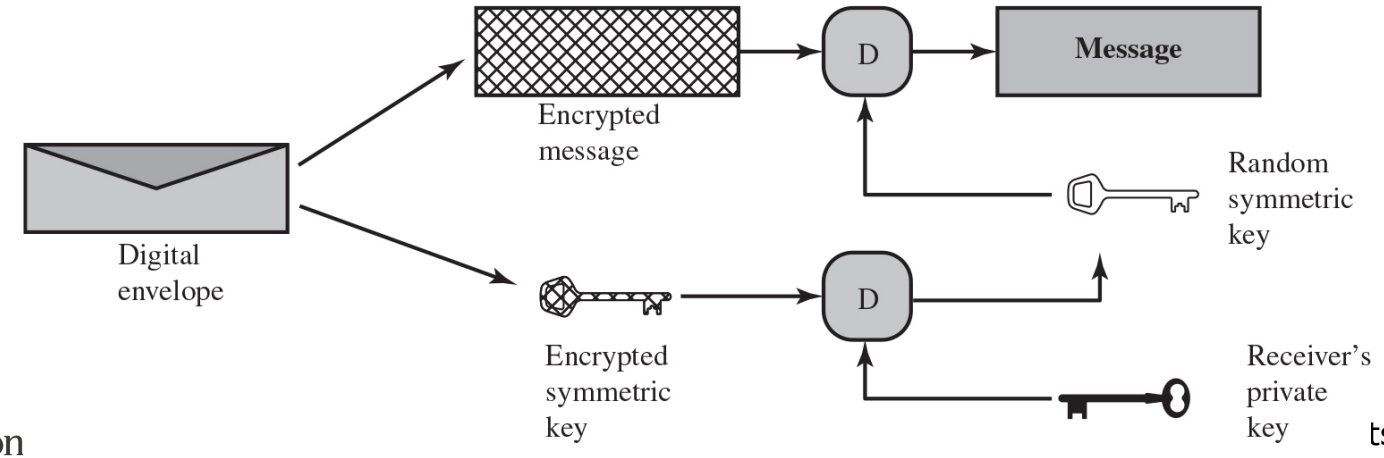
* **Signed Digest** – A cryptographic hash (digest) of the message, signed with the sender's private key.

Digital Envelopes Technique

The public-key encryption is used to protect a symmetric key, which can be used to protect a message **without needing to first arrange for sender and receiver to have the same secret key**.
It is called digital envelope, **which is the equivalent of a sealed envelope containing an unsigned letter**.



(a) Creation of a digital envelope



(b) Opening a digital envelope

RSA Public-Key Encryption

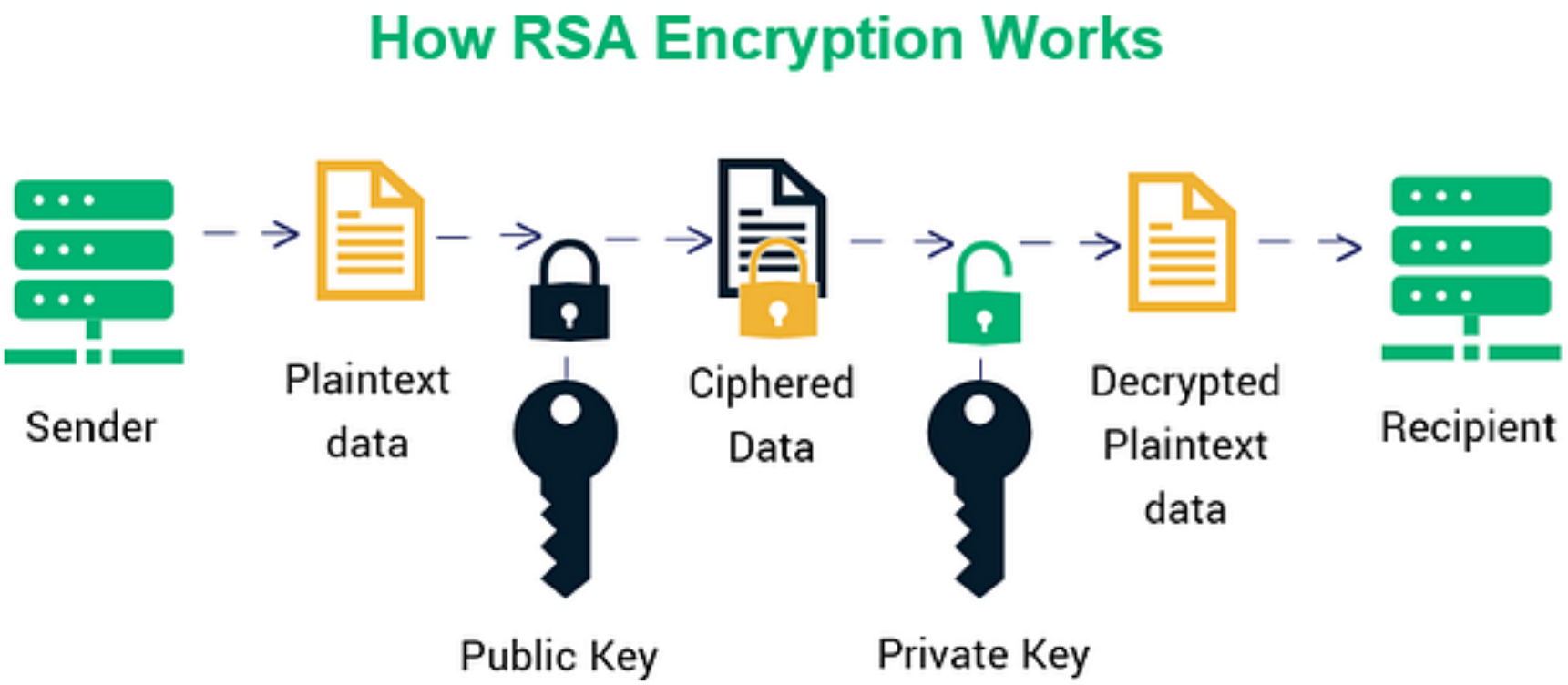
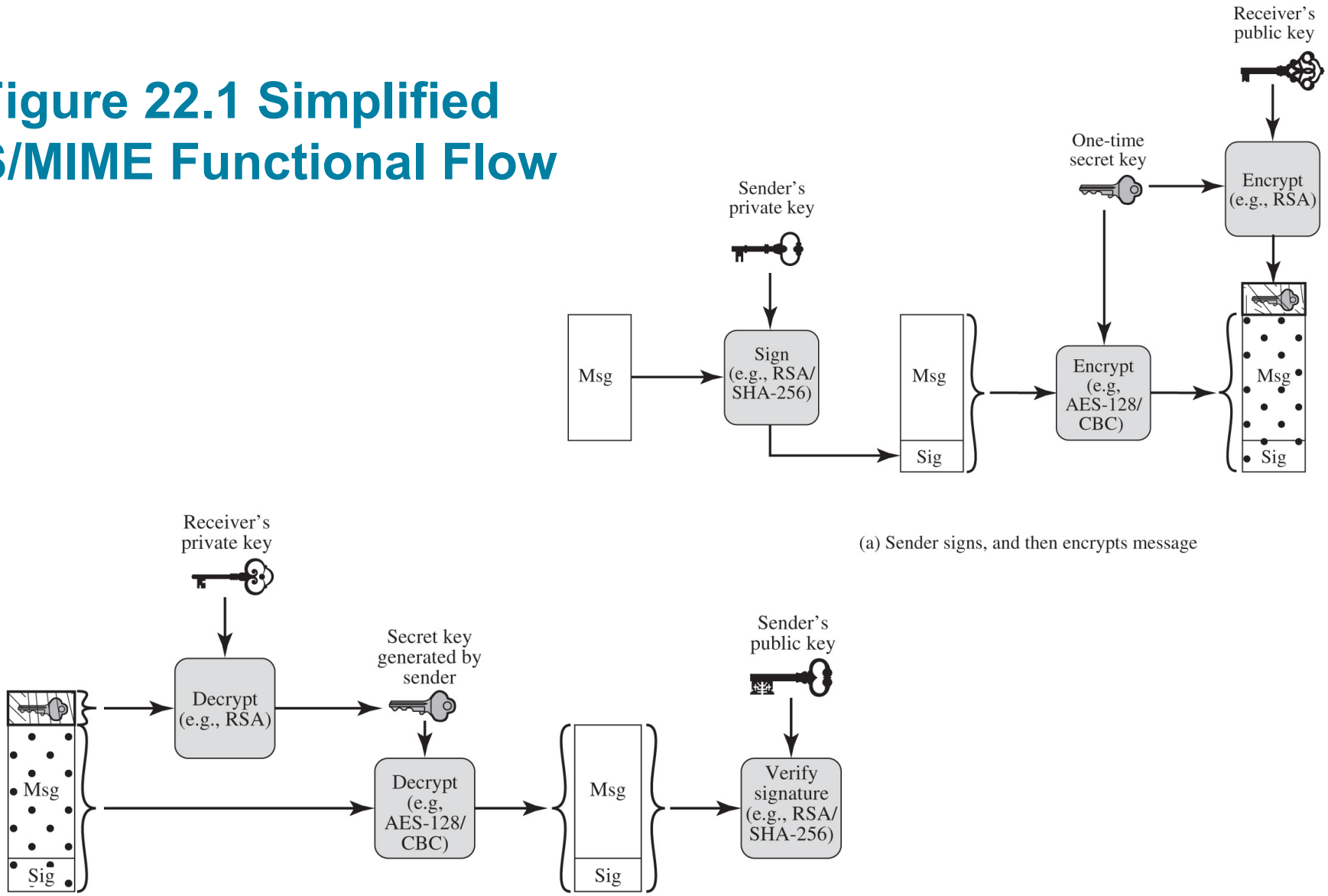
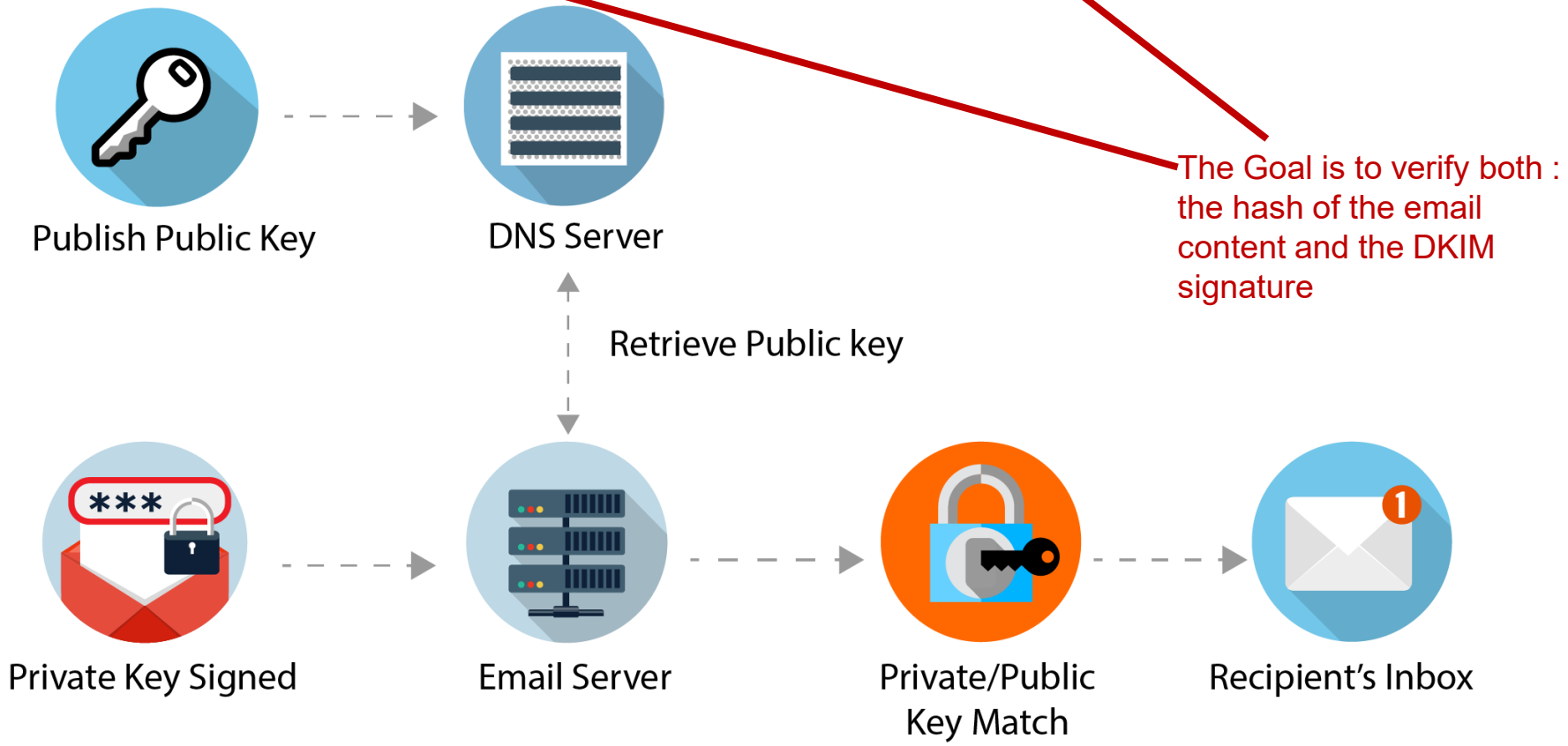


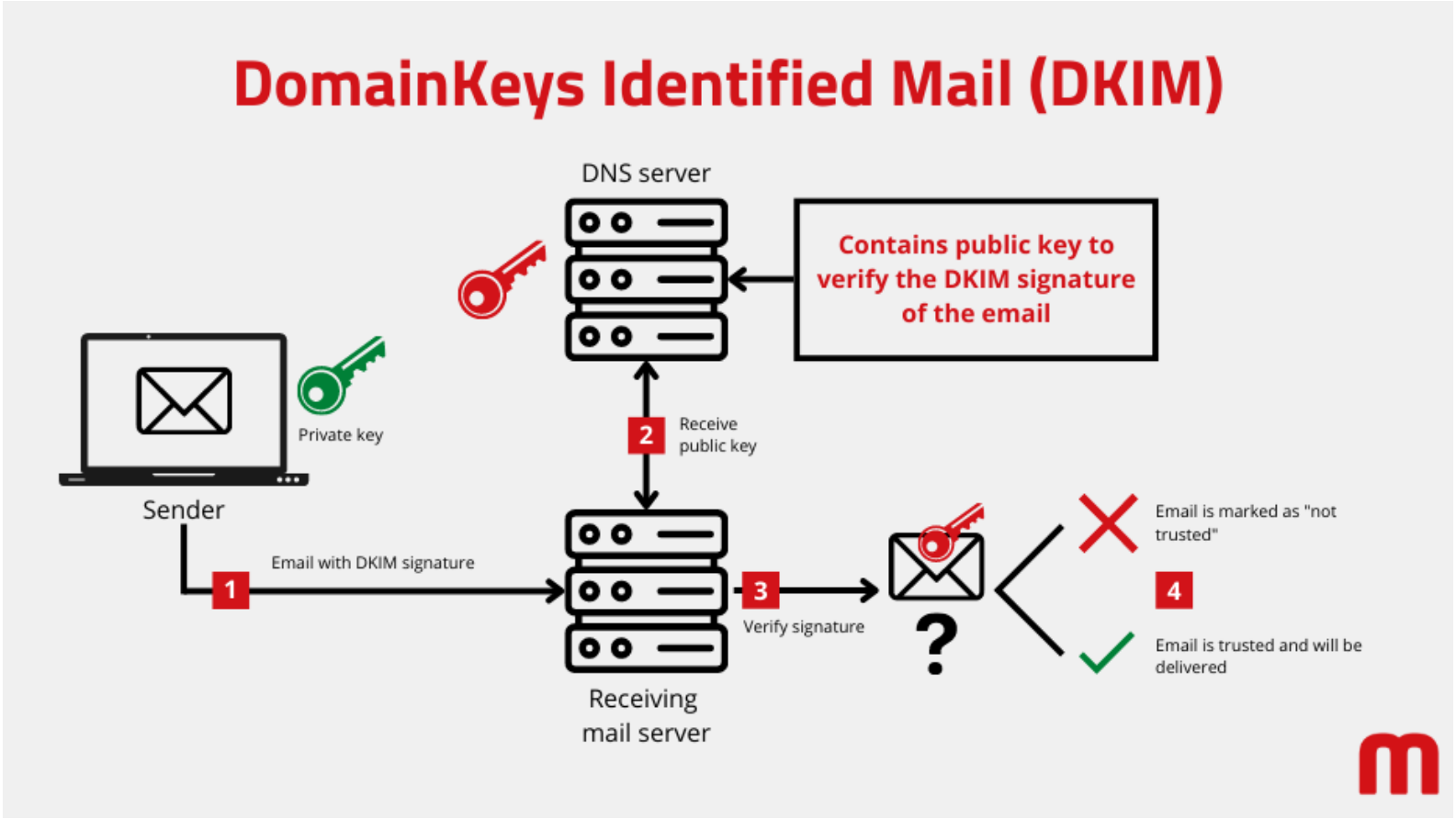
Figure 22.1 Simplified S/MIME Functional Flow



DomainKeys Identified Mail (DKIM)

- DKIM is an email authentication method that allows email recipients to verify that ***an email was sent from an authorized mail server and was not altered in transit***.





How Bob's Email Server Verifies DKIM (Step by Step)

📩 **Alice (alice@example.com) sends an email to Bob (bob@example.org).**

Bob's email server will verify DKIM as follows:

🔹 Step 1: Extract the DKIM Signature

Bob's email server finds the **DKIM-Signature** header in the received email.

Example:

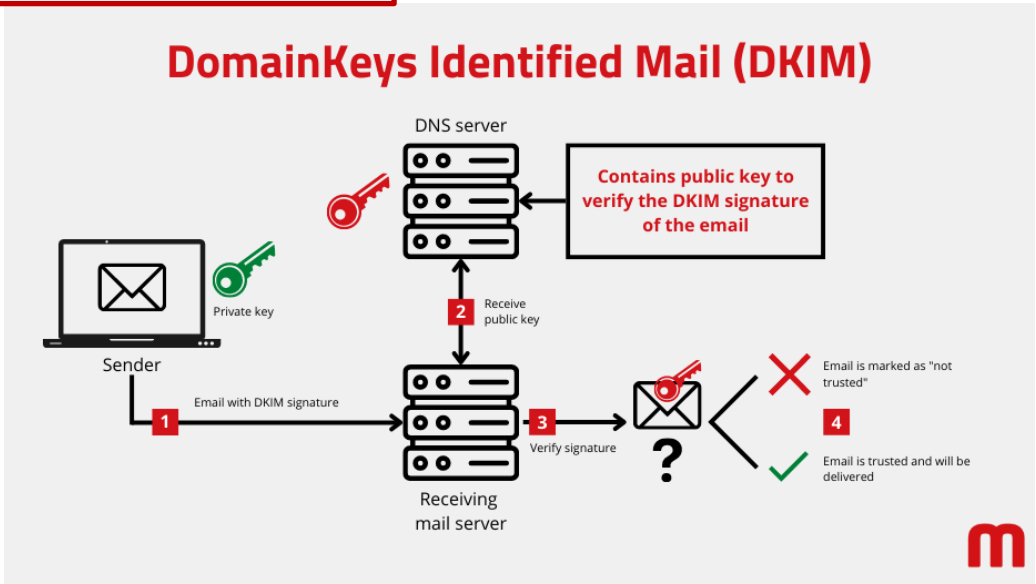
makefile — Copy — Edit

```
DKIM-Signature: v=1; a=rsa-sha256; d=example.com; s=selector1;
h=from:subject:date;
bh=abc123hashedmessage;
b=MIIBIjANBgkqhkiG9...
```

🧿 **Key parts of the signature:**

- `d=example.com` → The domain that signed the email.
- `s=selector1` → The selector used to find the public key.
- `bh=abc123hashedmessage` → The hash of the email content.
- `b=MIIBIjANBgkqhkiG9...` → The actual **DKIM signature**.

The Goal is to verify both : the hash of the email content and the DKIM signature



🔹 Step 2: Retrieve the Public Key from DNS

Bob's server does a **DNS lookup** to get Alice's public key.

It queries:

```
selector1._domainkey.example.com
```

Copy Edit

And receives a TXT record like this:

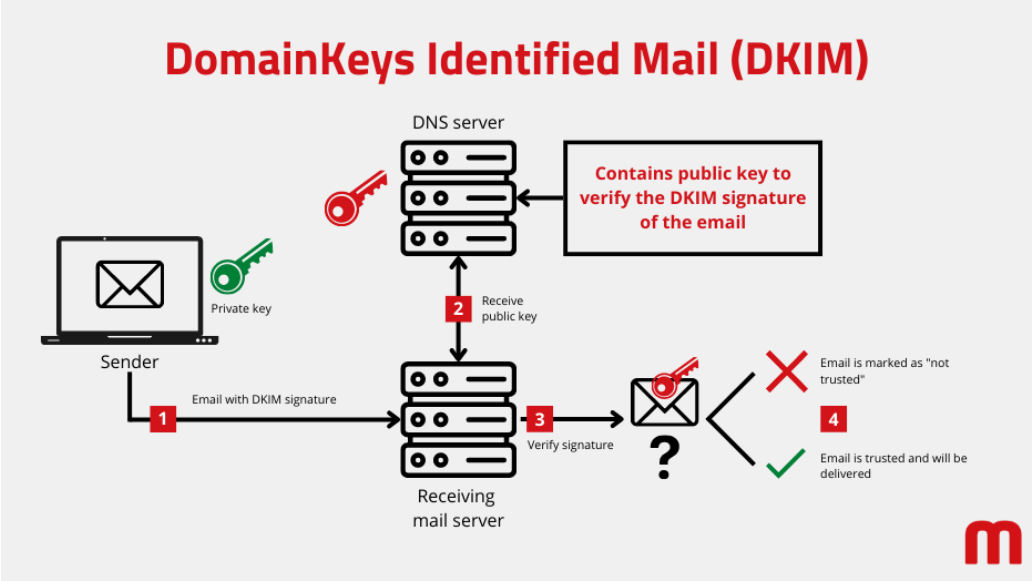
ini

```
v=DKIM1; k=rsa; p=MIIBIjANBgkqhkiG9...
```

Copy Edit

- `p=...` → This is the **public key** that Bob's server will use for verification.

Retrieves the public key from the DNS using the S Selector



Step 3: Recompute the Hash

Bob's email server **takes the original email headers** (like `From`, `Subject`, `Date`), then:

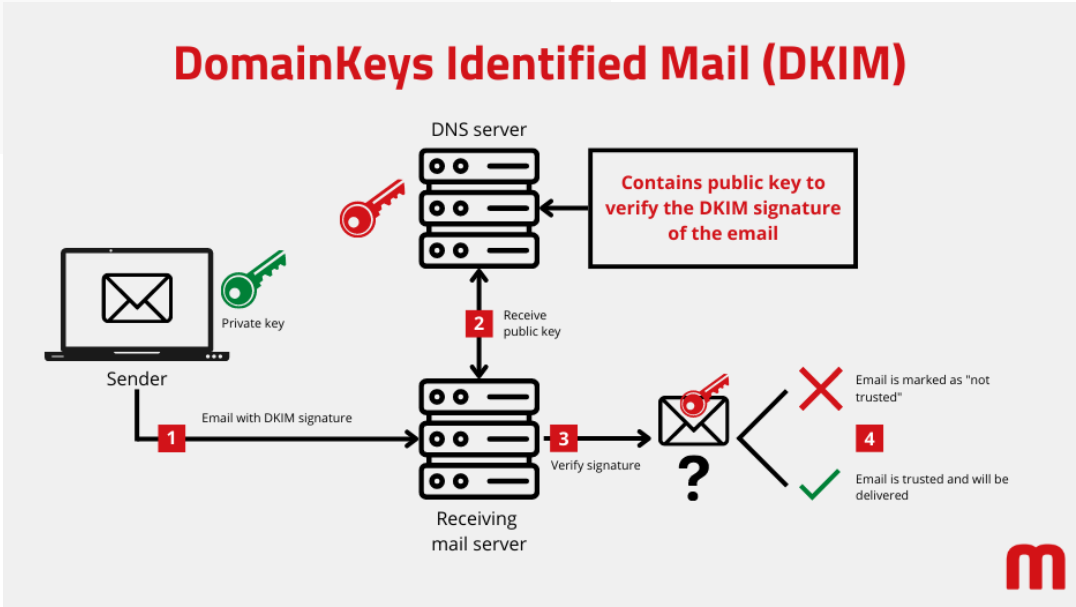
1. **Hashes them** using the same algorithm (`rsa-sha256`).
2. Compares this newly computed hash with the **bh=abc123hashedmessage** in the DKIM header.
   - If they match, ✅ the email content is **unchanged**.
   - If they don't, ❌ the email may have been modified in transit.

```
makefile

DKIM-Signature: v=1; a=rsa-sha256; d=example.com; s=selector1;
h=from:subject:date;
bh=abc123hashedmessage;
b=MIIBIjANBgkqhkiG9...
```

Verifying bh, the hash of the email content

Bh → hash of the email content



Step 4: Verify the Signature with the Public Key

1. Bob's server takes the **DKIM signature** (`b=...`) from the email.
2. It **decrypts** it using Alice's **public key** (from DNS).
3. If the decrypted signature matches the recomputed hash, ✅ **DKIM passes** (email is authentic).
4. If it **does not match**, ❌ **DKIM fails** (email might be forged or tampered with).

makefile

```
DKIM-Signature: v=1; a=rsa-sha256; d=example.com; s=selector1;
h=from:subject:date;
bh=abc123hashedmessage;
b=MIIBIjANBgkqhkiG9...
```

Verifying b, the DKIM signature

Both Step 3 and 4 have to pass

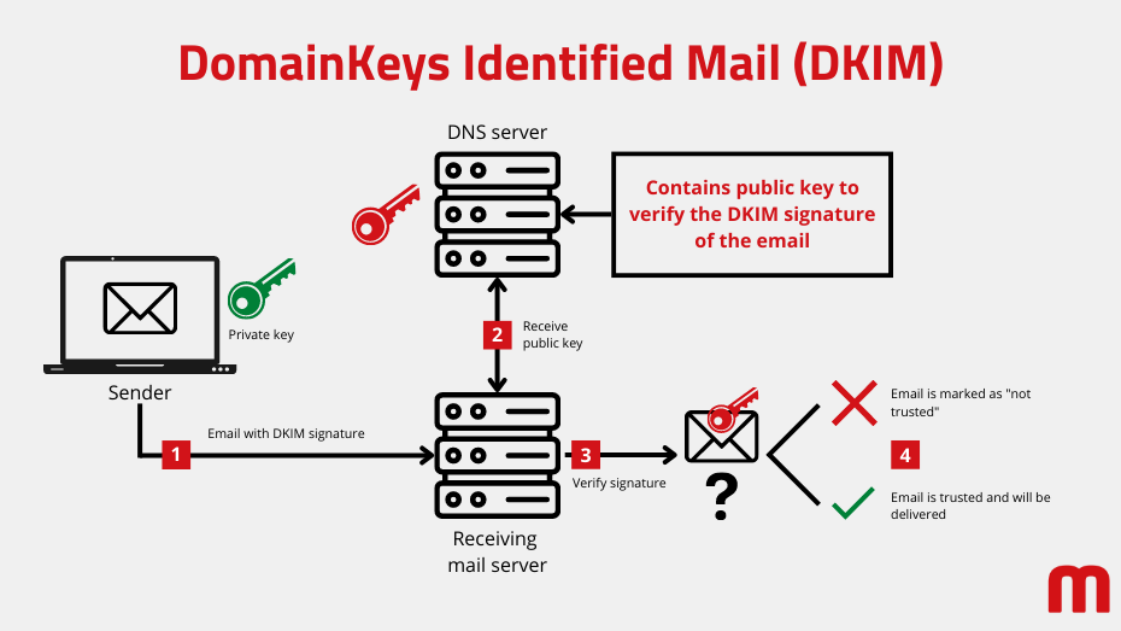
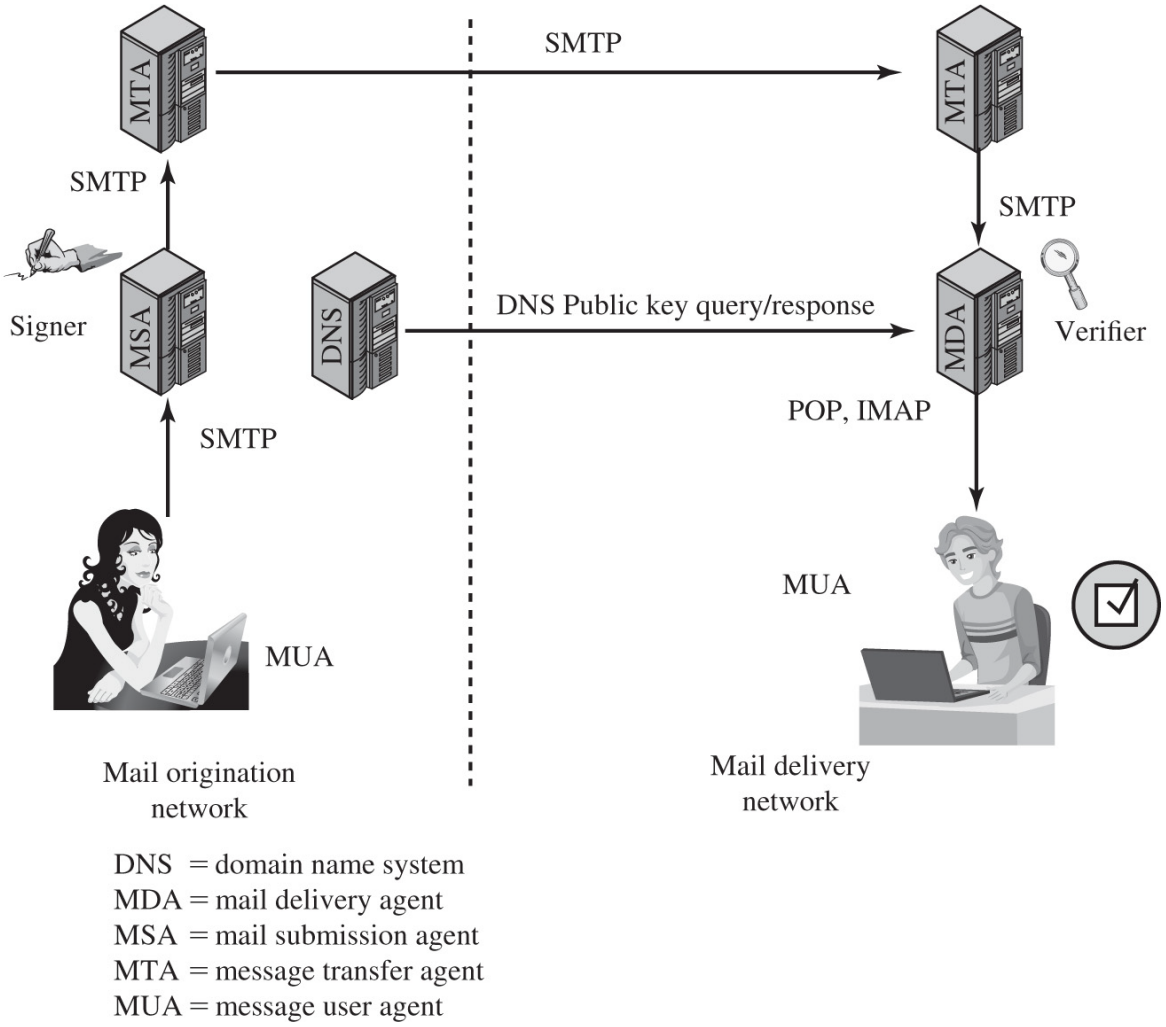


Figure 22.3 Simple Example of DKIM Deployment



SECURE SOCKETS LAYER (SSL) AND TRANSPORT LAYER SECURITY (TLS)

- **TLS (Transport Layer Security)** and **SSL (Secure Sockets Layer)** are cryptographic protocols designed to provide **secure communication over the internet**. While they serve the same purpose, TLS is the **modern, more secure replacement for SSL**.

- **SSL** was the first widely used protocol for encrypting internet communications.
  - It was developed by Netscape in the mid-1990s.
  - It has multiple vulnerabilities and is now considered obsolete. It is no longer used due to its security weakness

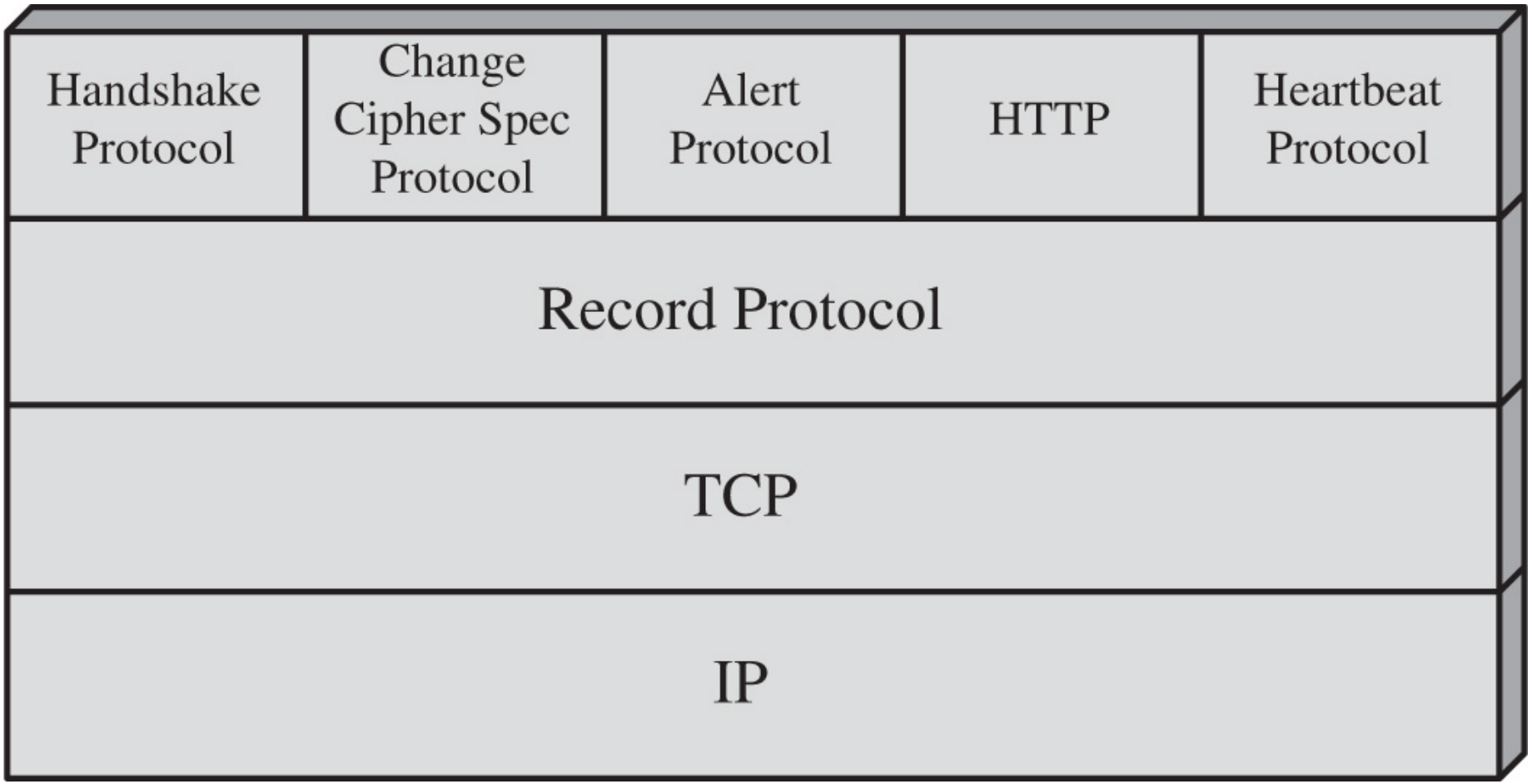
- **TLS** is the successor to SSL, offering stronger encryption and security improvements.
  - **One of the most widely used security services**
  - General-purpose service implemented as a set of protocols that rely on TCP
  - Subsequently became Internet standard RFC4346: Transport Layer Security (TLS)

  Two implementation choices:

  - Provided as part of the underlying protocol suite
  - Embedded in specific packages

Figure 22.4 SSL/TLS Protocol Stack

This stack of protocols are used when we open a session over the internet. **The main function is to enhance the security level to any kind of communication.**



TLS Concepts

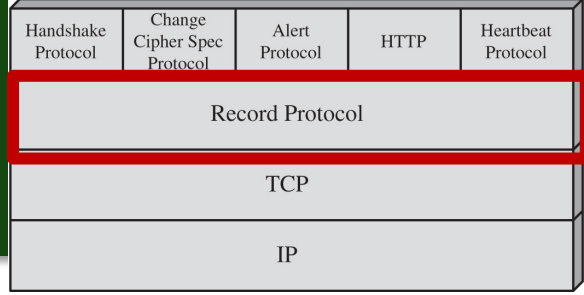
TLS Connection

- A transport (in the OSI layering model definition) that provides a suitable type of service
- **Peer-to-peer relationships**
- Transient
- Every connection is associated with one session

TLS Session

- **An association between a client and a server**
- Created by the Handshake Protocol
- Define a set of cryptographic security parameters
- Used to avoid the expensive negotiation of new security parameters for each connection

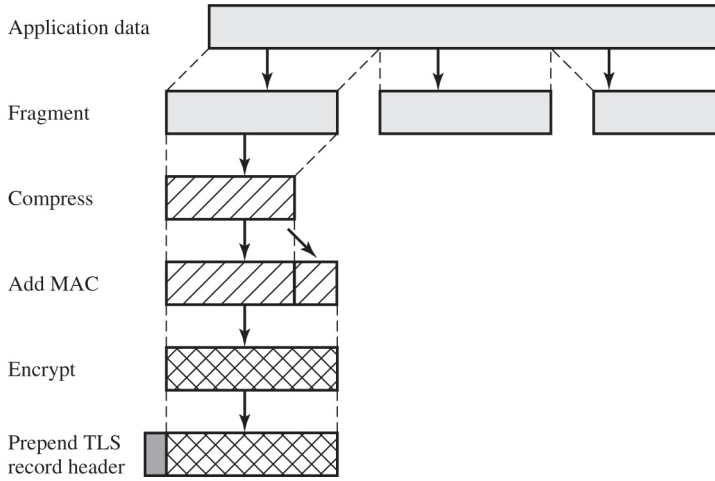
TLS RECORD PROTOCOL



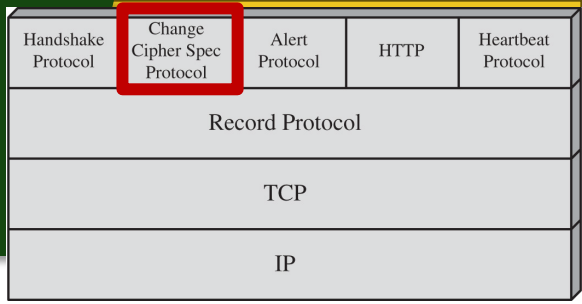
The TLS record protocol provides secure, reliable transmission of data over a network by ensuring confidentiality, integrity, and authenticity.

The TLS record protocol secures data communications by:

- Breaking data into records.
- Optionally compressing the data.
- Appending a MAC for data integrity.
- Encrypting the data to ensure confidentiality.
- Packaging the record with a header that indicates its type and size.



CHANGE CIPHER SPEC PROTOCOL

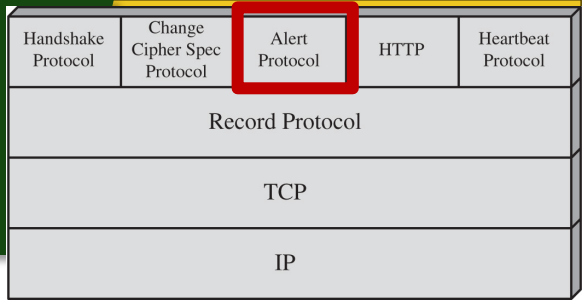


Handles Pending Requests

- One of four TLS specific protocols that use the TLS Record Protocol
- Is the simplest
- **Consists of a single message which consists of a single byte with the value 1**
- **Sole purpose of this message is to cause pending state to be copied into the current state. Hence updating the cipher suite in use**

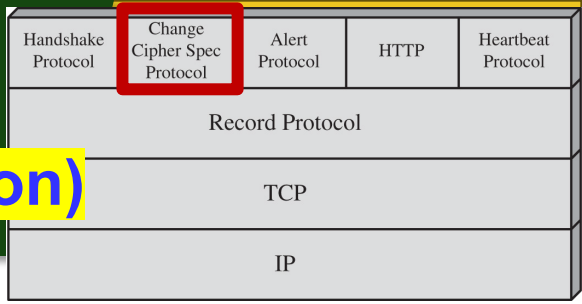
ALERT PROTOCOL

Sends alerts related to compression or encryption



- **Conveys TLS-related alerts to peer entity**
  - **Alert messages are compressed and encrypted**
    - Each message consists of two bytes:
      - First byte takes the value warning (1) or fatal (2) to convey the severity of the message
        - If the level is fatal, TSL immediately terminates the connection
        - Other connections on the same session may continue, but no new connections on this session may be established
      - Second byte contains a code that indicates the specific alert

HANDSHAKE PROTOCOL



Trust between Client & Server (Authentication)

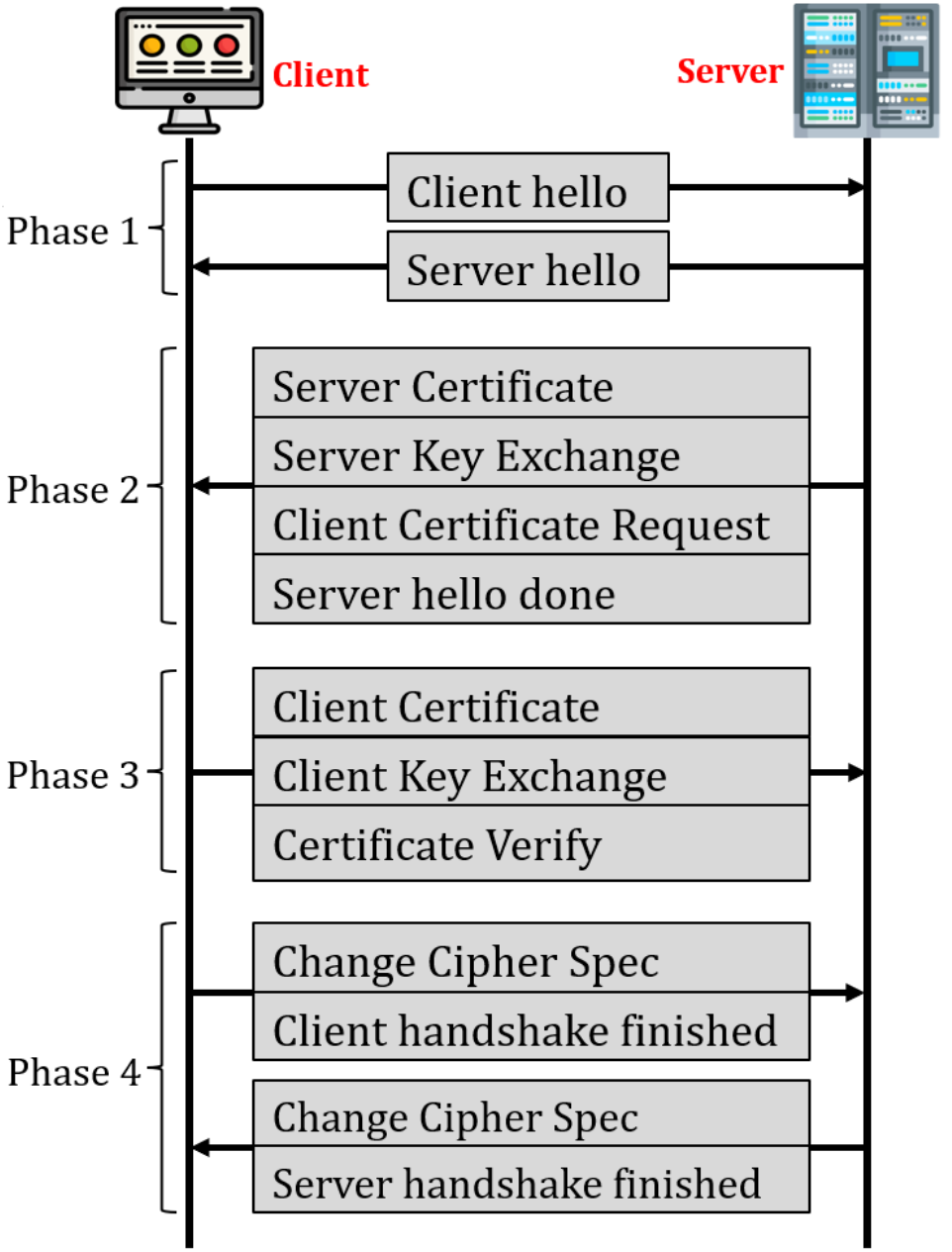
- Most complex part of TLS
- **Is used before any application data are transmitted**

- **Allows server and client to:**
- **Authenticate each other**
- **Negotiate encryption and MAC algorithms**
- **Negotiate cryptographic keys to be used**

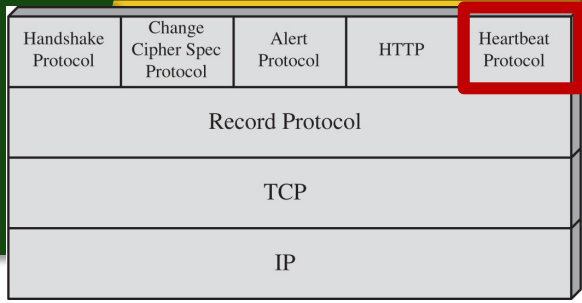
- Comprises a series of messages exchanged by client and server
- Exchange has four phases

HANDSHAKE PROTOCOL ACTION

Phase 1
Establishing Security Capabilities
Phase 2
Server Authentication and Key Exchange
Phase 3
Client Authentication and Key Exchange
Phase 4
Finalizing the Handshake Protocol



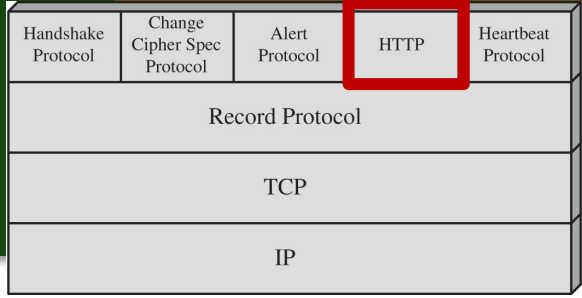
HEARTBEAT PROTOCOL



Ensures the two parties are still alive

- **A periodic signal generated by hardware or software to indicate normal operation or to synchronize other parts of a system**
- Typically used to monitor the availability of a protocol entity
- Runs on top of the TLS Record Protocol
- Use is established during Phase 1 of the Handshake Protocol
- Each peer indicates whether it supports heartbeats
- **Serves two purposes:**
  - **Assures the sender that the recipient is still alive**
  - **Generates activity across the connection during idle periods**

HTTPS (HTTP OVER SSL)

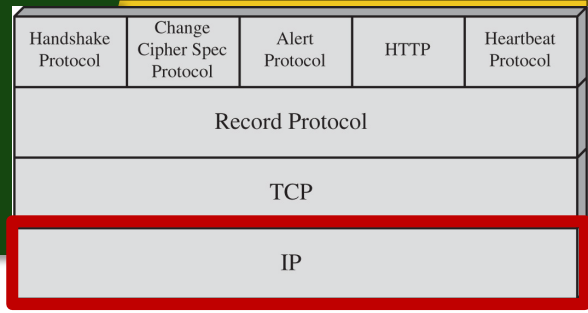


HTTPS not HTTP

- **Combination of HTTP and SSL to implement secure communication between a Web browser and a Web server**
- Built into all modern Web browsers
  - Search engines do not support HTTPS
  - URL addresses begin with https://
- Agent acting as the HTTP client also acts as the TLS client
- Closure of an HTTPS connection requires that TLS close the connection with the peer TLS entity on the remote side, which will involve closing the underlying TCP connection

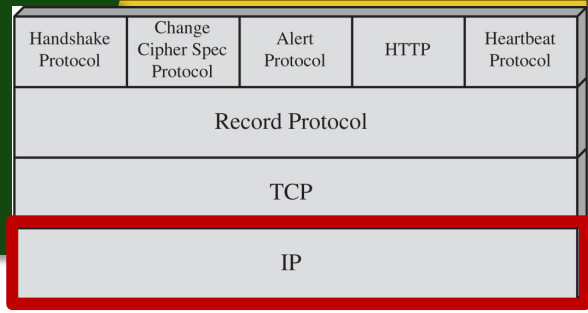
IP SECURITY (IPSEC)

Additional Security layer



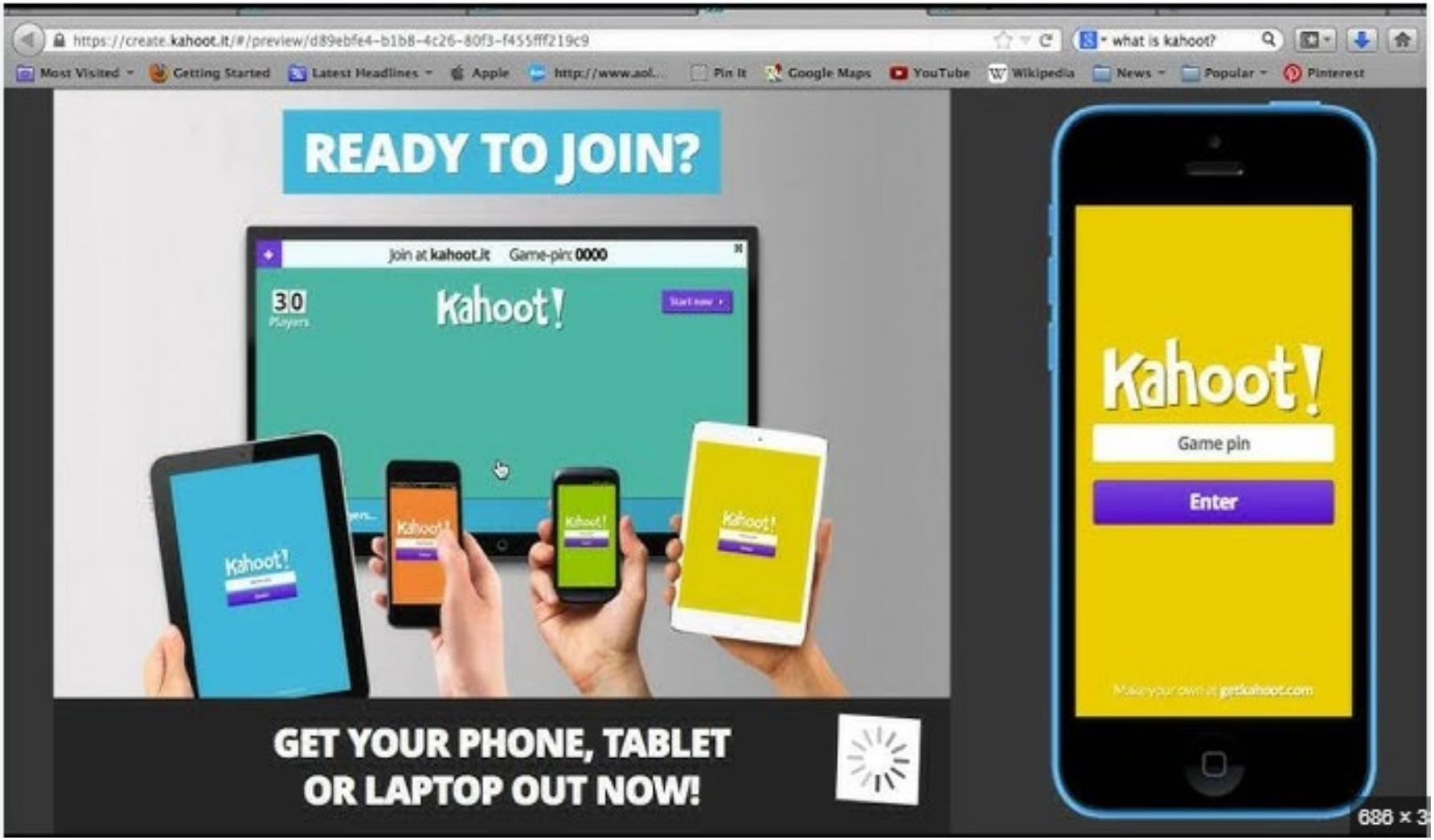
- Various application security mechanisms
  - S/MIME, Kerberos, SSL/HTTPS
- Security concerns cross protocol layers
- Would like security implemented by the network for all applications
- **Authentication and encryption** security features included in next-generation IPv6
- Also usable in existing IPv4

BENEFITS OF IPSEC



- **When implemented in a firewall or router, it provides strong security to all traffic crossing the perimeter**
- In a firewall it is resistant to bypass
- Below transport layer, hence transparent to applications
- Can be transparent to end users
- Can provide security for individual users
- Secures routing architecture

Kahoot!


READY TO JOIN?
Join at kahoot.it Game-pin: 0000
30
Players
Kahoot!
Start now
Kahoot!
Game pin
Enter
GET YOUR PHONE, TABLET
OR LAPTOP OUT NOW!

1. MIME stands for:
a) Multipurpose Internet Message Exchange
b) Mail Internet Message Extension
c) Multipurpose Internet Mail Extensions
d) Multiprotocol Internet Mail Encryption

2. What is the primary purpose of S/MIME?
a) To compress email attachments
b) To provide security enhancements for email communication
c) To filter spam emails
d) To increase email storage capacity

3. PKCS#7 is a standard used for:
a) Digital signatures
b) Cryptographic message syntax
c) Email header encryption
d) Secure browsing

4. Which of the following is NOT a PKCS#7 MIME format?
a) SignedData
b) EnvelopedData
c) CompressedData
d) HashedData

5. What does DKIM help verify?
a) The sender of an email is authorized
b) The recipient's email client compatibility
c) The email storage format
d) The compression ratio of an email attachment

6. The Secure Sockets Layer (SSL) is primarily used for:
a) Securing data transmission over TCP
b) Encrypting stored files
c) Protecting local files
d) Compressing network packets

7. TLS stands for:
a) Transport Layer Security
b) Transmission Link Security
c) Trusted Layer System
d) Transport Line Security

8. The TLS Handshake Protocol is responsible for:
a) Establishing a secure session between a client and a server
b) Terminating all active connections
c) Encrypting only the first packet of data
d) Managing packet compression

9. The Change Cipher Spec Protocol in TLS is used for:
a) Encrypting TLS header information
b) Informing the peer to update encryption keys
c) Compressing encrypted data
d) Verifying certificate expiration

10. A TLS alert message with a severity level of "fatal" results in:
a) The session continuing as normal
b) The connection being immediately terminated
c) The client attempting to reconnect
d) The server downgrading to SSL

11. Which TLS protocol monitors connection availability?
a) Handshake Protocol
b) Change Cipher Spec Protocol
c) Heartbeat Protocol
d) Alert Protocol

12. The Heartbleed exploit affected which protocol?
a) MIME
b) DKIM
c) TLS
d) IPv6

13. HTTPS is a combination of:
a) HTTP and SSL/TLS
b) HTTP and MIME
c) SSL and FTP
d) TLS and SMTP

14. What is the primary goal of IPsec?
a) To enhance IP-based security for applications
b) To replace SSL for web security
c) To compress TCP/IP packets
d) To improve DNS resolution

15. When implemented in a firewall, IPsec provides:
a) Security for all traffic passing through
b) Encryption for only outgoing traffic
c) Security only for IPv6 networks
d) Automatic key exchange without user authentication

16. What are the two main functions of IPsec?
a) Packet filtering and NAT traversal
b) Authentication and encryption
c) DNS resolution and protocol compression
d) Firewall monitoring and alert generation

Summary

- Secure E-mail and S/MIME
  - MIME
  - S/MIME
- DomainKeys identified mail
  - Internet mail architecture
  - DKIM strategy
- SSL and TLS
  - TLS architecture
  - TLS protocols
  - TLS attacks
  - SSL/TLS attacks
- HTTPS
  - Connection institution
  - Connection closure
- IPv4 and IPv6 security
  - IP security overview
  - The scope of IPsec